INNOPOLIS UNIVERSITY

SNE MASTER
TRACK 2026

COURSE: [S26] Computer Forensics and Incident Response

CCF Lab 4 Report

On

SIEM Deployment for Threats Detection

Student Name: HASIB AL TAHSIN

Application ID: 187623
Student ID: 24

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 14.04.2026

INDEX

Task 1…… 3-11

Task 2…… 11-25

Task 3…… 25-56

Task 4…… 56-71

References……. 71-71

Task 1 - Wazuh deployment

Install Wazuh SIEM solution in any way (deployment scenario) that you prefer. Consider a possible lab environment:

- **Wazuh server (could be as a part of Elastic stack)**
- **Wazuh agents (workers under monitoring, at least one node, but more is better)**
- **Attacker host (not mandatory, but much better to have)**

It's up to you where to create the lab environment and its nodes (locally or in the cloud). But make sure that if you are going to simulate an any attacks/threats or exploit vulnerabilities, your attacker host and workers are in isolated network. Thus, it's recommended do not perform any illegal actions it the cloud. This is in your area of responsibility, be very careful and attentive!

Bonus: prepare a different agents: Windows and Linux based

Bonus: substitute Wazuh by other SIEM solution that has the comparable functionality. Show a PoC why it's a worthy alternative

Solution:

The main objective of this task is to prepare *Kali Linux* as a centralized security monitoring server.

Prerequisites and System Updates

Before running heavy software like *SIEM*, it is mandatory to keep the system up-to-date and install the required dependencies.

Docker and Tools Installation

I run *Wazuh* through a Docker container, which is the best method for memory management and isolation.

```
┌──(kali㉿hasib)-[~]
└─$ sudo apt install -y docker.io docker-compose curl wget git
```

Output:

```
┌──(kali㉿hasib)-[~]
└─$ sudo apt install -y docker.io docker-compose curl wget git
[sudo] password for kali:
docker.io is already the newest version (28.5.2+dfsg3-2).
docker-compose is already the newest version (2.40.3-2).
curl is already the newest version (8.19.0-1).
curl set to manually installed.
wget is already the newest version (1.25.0-2).
git is already the newest version (1:2.53.0-1).
git set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 498
```

- ***docker.io:*** This is the core *Docker* engine that runs containers.
- ***docker-compose:*** This is needed to manage multiple containers (e.g. *Indexer, Manager, Dashboard*) together.
- ***curl, wget, git:*** These tools are used to download the *Wazuh* repository and configuration files.

Docker Service Configuration and Permission Setup

After installation, the service is enabled and permission to use Docker without root privileges is ensured.

The following command loads and starts the *Docker* service into memory.

```
┌──(kali㉿hasib)-[~]
└─$ sudo systemctl start docker
```

The following command ensures that Docker starts automatically even after a restart.

```
┌──(kali㉿hasib)-[~]
└─$ sudo systemctl enable docker
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker
```

This command adds the current user (*hasib*) to the Docker group, eliminating the need to issue the *sudo* command repeatedly.

```
┌──(kali㉿hasib)-[~]
└─$ sudo usermod -aG docker $USER
```

The following command makes group changes take effect immediately without rebooting the system.

```
┌──(kali㉿hasib)-[~]
└─$ newgrp docker
```

Installation Verification

Version has been checked to ensure that the system is ready. The analysis according to my terminal output is given below:

```
┌──(kali㉿hasib)-[~]
└─$ docker --version
Docker version 28.5.2+dfsg3, build 9cc6dea35e9a963f281434761c656fba4ac43aed
```

Output: *Docker version 28.5.2+dfsg3*

The latest and stable version of Docker is installed on the system. This confirms that the containerized environment is now ready to work.

Docker-Compose Version Check:

Output: *Docker version 28.5.2+dfsg3*

The latest and stable version of Docker is installed on the system. This confirms that the containerized environment is now ready to work.

Docker-Compose Version Check:

```
┌──(kali㉿hasib)-[~]
└─$ docker-compose --version
Docker Compose version 2.40.3-2
```

Output: Docker Compose version 2.40.3-2

The Compose version is suitable for processing the *docker-compose.yml* files of *Wazuh.* With this, I can later launch the entire *SIEM* stack in one command.

Agent Environment Preparation (VirtualBox)

A hypervisor is required to run the agent or target machine (Ubuntu).

Command:

```
┌──(kali㉿hasib)-[~]
└─$ sudo apt install -y virtualbox virtualbox-ext-pack
```

Virtual Box helps create an isolated lab to test network communication between *Wazuh* server (*Kali*) and agent (*Ubuntu*).

License Agreement: By selecting '*OK*' and '*Yes*' here, I have enabled *USB 2.0/3.0* support and other enhancement features in the *Ubuntu* guest machine.

Lab Environment Architecture

There are three machines in this lab:

Machine	OS	Role	Network
Wazuh Server	Kali Linux (Docker)	SIEM/XDR Server	Host Machine

Machine	OS	Role	Network
Linux Agent	Ubuntu 22.04 (VirtualBox VM)	Monitored Host	Host-only Network
Attacker	Kali Linux (same host)	Attack Simulation	Host Machine

Wazuh Docker Deployment

In this step, I deployed the *Wazuh* stack on a single-node architecture. It consists of three main components: *Indexer, Manager*, and *Dashboard*.

Clone the repository and verify the file structure

We downloaded the official *Wazuh* Docker configuration files from GitHub.

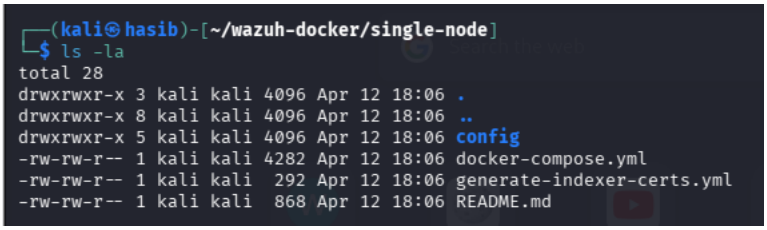
Command:

git clone https://github.com/wazuh/wazuh-docker.git -b v4.7.3



Here I have made sure that I am using a specific and stable version by using *-b v4.7.3*.

To see what files have been created:



docker-compose.yml: This is the master configuration file that controls the images, networks, and volumes of the containers.

config/: This is where the internal configuration files of the *Wazuh* server are located.

SSL Certificates Generation (Security Layer)

SSL certificates are required to encrypt communication between each component of the *Wazuh* stack.

Command:

docker-compose -f generate-indexer-certs.yml run --rm generator

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ docker-compose -f generate-indexer-certs.yml run --rm generator
```

My output shows that the *Admin*, *Indexer, Server*, and *Dashboard* certificates have been created.

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ docker-compose -f generate-indexer-certs.yml run --rm generator
WARN[0000] /home/kali/wazuh-docker/single-node/generate-indexer-certs.yml: th
e attribute `version` is obsolete, it will be ignored, please remove it to av
oid potential confusion
[+] Creating 1/1
 ✔ Network single-node_default  Create ...                                0.1s
[+] Running 5/5
 ✔ generator Pulled                                                      10.0s
   ✔ edaedc954fb5 Pull complete                                           6.2s
   ✔ 573f4d11a520 Pull complete                                           7.8s
   ✔ 8f200922197d Pull complete                                           7.8s
   ✔ 55a86de68c5c Pull complete                                           7.8s
The tool to create the certificates exists in the in Packages bucket
12/04/2026 22:08:20 INFO: Admin certificates created.
12/04/2026 22:08:20 INFO: Wazuh indexer certificates created.
12/04/2026 22:08:20 INFO: Wazuh server certificates created.
12/04/2026 22:08:20 INFO: Wazuh dashboard certificates created.
Moving created certificates to the destination directory
Changing certificate permissions
Setting UID indexer and dashboard
Setting UID for wazuh manager and worker
```

Why it's important: Without a certificate, the dashboard won't open in the browser and the data won't reach the indexer. This protects the *SIEM* infrastructure from man-in-the-middle (*MITM*) attacks.

Launching the Wazoo Stack

After all the preparations, I launched the entire environment.

Command:

docker-compose up -d

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ docker-compose up -d
```

Output:



This command downloads (*pulls*) all the required Docker images (*Manager, Indexer, Dashboard*) from the cloud. Then creates the required volumes where the security logs and configuration data will be permanently stored. *-d (Detached Mode*) which ensures that the server will continue to run in the background even if you close my terminal.

Status Verification

Service Name	Status	Ports	Role
Wazuh Indexer	Up (healthy)	9200	This stores and searches all log data.
Wazuh Manager	Up (healthy)	1514, 1515, 55000	This receives data from agents and analyzes it.
Wazuh Dashboard	Up (healthy)	443 (HTTPS)	This is the graphical interface where I can view alerts.

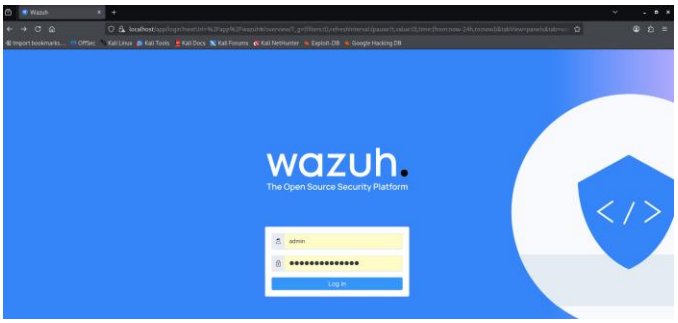
My output shows ports *443->5601* and *9200* open. This means that the firewall or network configuration is fine and I can now access the *SIEM* from the browser.

Dashboard Access and Interface Verification

After starting the *SIEM* server, it is very important to verify its graphical user interface (*GUI*) or dashboard. This is the place from where a security analyst monitors the entire network.

Secure Login (Authentication)

The dashboard is accessed via *HTTPS* protocol using *https://localhost* in the browser.

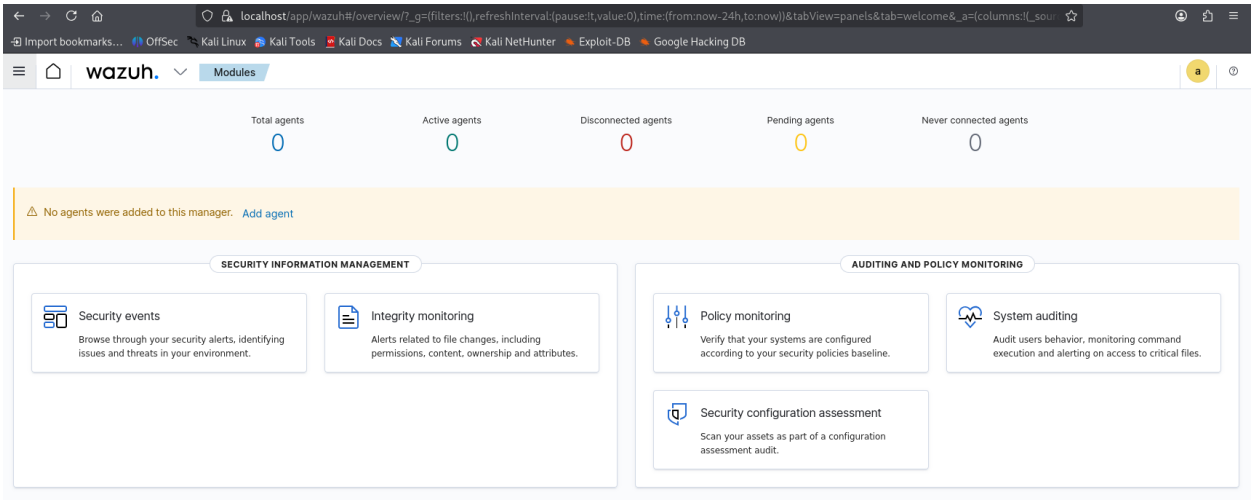


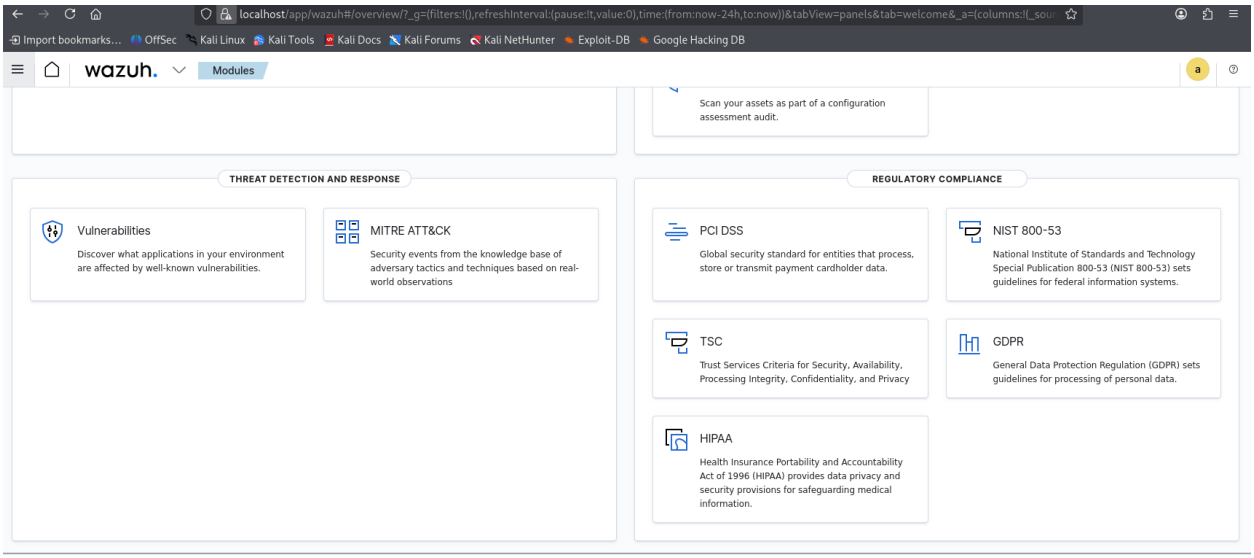
Authentication was successfully completed using the default credentials (*admin / SecretPassword*).

I bypassed the *SSL* Warning that came up initially by clicking "*Accept the Risk*" because I am using my own self-signed certificate.

Dashboard Overview Analysis

The interface I see after logging in is *Wazuh*'s central hub.





Agent Summary:

Currently showing *Total agents: 0*. This means that the server *(Kali)* has been created, but no clients (Ubuntu) have been added yet. In the next step, our task will be to increase this number**.**

Module Introduction:

- **Security Events:** Real-time attacks or logs can be viewed from here.
- **Integrity Monitoring (FIM):** Alerts will be sent here if any files in the system are changed.
- **Vulnerabilities:** If there is any security flaw in the agent software, it will be detected here.
- **Regulatory Compliance:** I can verify whether my system is compliant with *PCI DSS*, *GDPR* or *NIST* standards or not.

Verification Command (Server Side):

The following command is used to re-check whether the dashboard container is running or not:

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ docker-compose ps

WARN[0000] /home/kali/wazuh-docker/single-node/docker-compose.yml: the attrib
ute `version` is obsolete, it will be ignored, please remove it to avoid pote
ntial confusion
NAME                              IMAGE                              COMMAND
                  SERVICE             CREATED             STATUS              PORTS
single-node-wazuh.dashboard-1     wazuh/wazuh-dashboard:4.7.3        "/entrypoint.sh
"                 wazuh.dashboard     44 seconds ago      Up 43 seconds       443/tcp, 0.0.0.0
:443->5601/tcp, [::]:443->5601/tcp
single-node-wazuh.indexer-1       wazuh/wazuh-indexer:4.7.3          "/entrypoint.sh
 open…"           wazuh.indexer       45 seconds ago      Up 43 seconds       0.0.0.0:9200->92
00/tcp, [::]:9200->9200/tcp
single-node-wazuh.manager-1       wazuh/wazuh-manager:4.7.3          "/init"
                  wazuh.manager       45 seconds ago      Up 43 seconds       0.0.0.0:1514-151
5->1514-1515/tcp, [::]:1514-1515->1514-1515/tcp, 0.0.0.0:514->514/udp, [::]:5
14->514/udp, 0.0.0.0:55000->55000/tcp, [::]:55000->55000/tcp, 1516/tcp
```

The output shows that the dashboard has loaded successfully and all security modules are visible in the Modules menu.

Task 2 - Configure the SIEM

Once Wazuh is installed, make familiar with its features and manager interface (it's not mandatory to describe in the report). Then configure agents ossec rules to work with a various Wazuh security processing possibilities. Rules types and their amount depends on how many different Wazuh features you are going to try in the next task.

Solution:

The goal of this task is to connect an *Ubuntu* virtual machine to *Wazuh Manager* (*Kali Linux*) so that all security events from *Ubuntu* can be seen in the dashboard.

Creating and Configuring Ubuntu Linux Agent

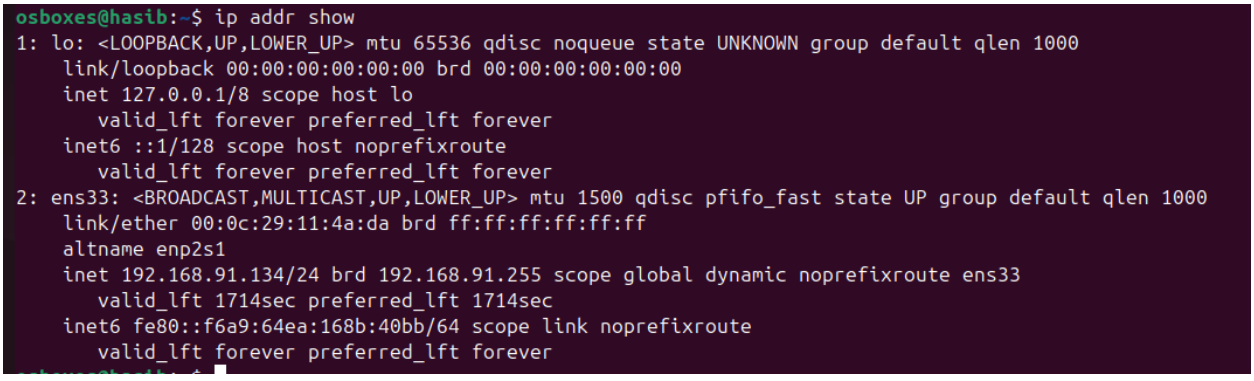
Virtual Lab Network Setup

Network configuration in *VirtualBox* is very important for smooth communication between the Agent and the Manager.

Host-only Adapter is selected from the *Ubuntu Agent VM* settings. This ensures that my *Kali Linux* and *Ubuntu* machines will be on the same private network and will be able to exchange data with each other.

Network Identification

Before installing the *Agent,* it is necessary to know the *IP* of *Wazuh Manager (Kali Linux)* which is done using the following command:



According to the output, the *IP* of *Kali Linux* on the *ens33* interface is *192.168.91.134.*

Endpoint Preparation and Repository Configuration

1. Internet Connectivity and Dependency Check

This is the first step to ensure communication with the server before installing the agent.

```
osboxes@hasib:~$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=40.9 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=38.9 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=39.4 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=128 time=39.3 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
rtt min/avg/max/mdev = 38.891/39.632/40.923/0.769 ms
osboxes@hasib:~$
```

The output shows *0% packet* loss, which means that the internet connection of the *Ubuntu VM* is fine.

Software Installation:

I installed curl which is essential for downloading files.

2. Adding GPG Key and Repository (Trust Building)

Before installing any security software, I need to make sure that its source is authentic.

Command:

curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg --no-default-keyring --keyring /usr/share/keyrings/wazuh.gpg –import

```
osboxes@hasib:~$ curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg --no-default-keyring --keyring /usr/share/keyrings/wazuh.gpg --import
```

Output:

```
osboxes@hasib:~$ curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg --no-default-keyring --keyring /usr/share/keyrings/wazuh.gpg --import

    Caution: You are using the Snap version of curl.
    Due to Snap's sandbox nature, this version has some limitations.
    For example, it may not be able to access hidden folders in your home directory
    or other restricted areas of the os.

    Which means you may encounter errors when using snap curl to download and execute some script.
    For those cases, you might want to use the native curl package.
    For details, see: https://github.com/boukendesho/curl-snap/issues/1

    To stop seeing this message, run the following command:
    $ curl.snap-acked

gpg: directory '/root/.gnupg' created
gpg: /root/.gnupg/trustdb.gpg: trustdb created
gpg: key 96B3EE5F29111145: public key "Wazuh.com (Wazuh Signing Key) <support@wazuh.com>" imported
gpg: Total number processed: 1
gpg:               imported: 1
```

The output shows that the public key "*Wazuh.com... imported*". This means that the *Ubuntu* system (*Agent*) will now accept *Wazuh*'s official packages as "*Trusted*".

Creating the repository file:

```
osboxes@hasib: $ echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main" | sudo tee /etc/apt/sources.list.d/wazuh.list
```

Output:

```
osboxes@hasib:~$ echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main" | sudo tee /etc/apt/sources.list.d/wazuh.list
deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main
```

Repository file:

I have successfully created the *wazuh.list* file. This tells the *Agent* from which internet address to find the latest versions of the *Wazuh Agent.*

Incorrect installation

1. Why is the Indexer not needed here?

According to my architecture, the *Wazuh Indexer* and *Dashboard* will be on *Kali Linux* (as the *Manager*). The *Ubuntu VM* acts as an Agent only.

```
osboxes@hasib:~$ sudo apt update
sudo apt install wazuh-indexer -y
Warning: The unit file, source configuration file or drop-ins of apt-news.service changed on disk. Run 'systemctl daemon-reload' to reload units.
Warning: The unit file, source configuration file or drop-ins of esm-cache.service changed on disk. Run 'systemctl daemon-reload' to reload units.
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Get:5 https://packages.wazuh.com/4.x/apt stable InRelease [17.3 kB]
Get:6 https://packages.wazuh.com/4.x/apt stable/main amd64 Packages [51.4 kB]
Fetched 68.6 kB in 1s (54.0 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
619 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  wazuh-indexer
0 upgraded, 1 newly installed, 0 to remove and 619 not upgraded.
Need to get 873 MB of archives.
After this operation, 1,102 MB of additional disk space will be used.
Get:1 https://packages.wazuh.com/4.x/apt stable/main amd64 wazuh-indexer amd64 4.14.4-1 [873 MB]
Fetched 873 MB in 27s (31.8 MB/s)
Selecting previously unselected package wazuh-indexer.
(Reading database ... 148052 files and directories currently instal
led.)
Preparing to unpack .../wazuh-indexer_4.14.4-1_amd64.deb ...
Running Wazuh Indexer Pre-Installation Script
```

```
Unpacking wazuh-indexer (4.14.4-1) ...
Setting up wazuh-indexer (4.14.4-1) ...
Running Wazuh Indexer Post-Installation Script
### NOT starting on installation, please execute the following stat
ements to configure wazuh-indexer service to start automatically us
ing systemd
 sudo systemctl daemon-reload
 sudo systemctl enable wazuh-indexer.service
### You can start wazuh-indexer service by executing
 sudo systemctl start wazuh-indexer.service
```

The mistake here is that I have installed *Wazuh-indexer* on *Ubuntu* or *Agent* machine.

Indexer is a huge package of about *873 MB* and requires a lot of *RAM* to run. This reduces the performance of the endpoint machine and complicates the lab architecture.

2. Version Mismatch

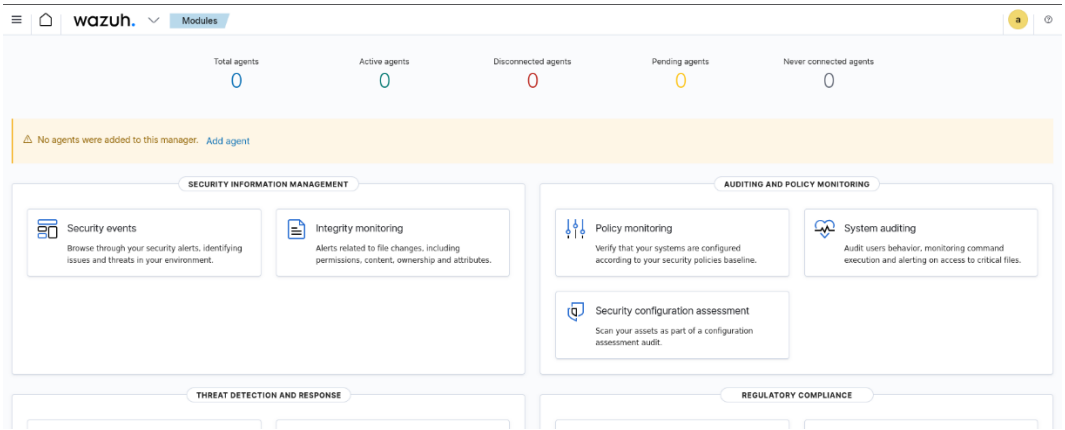
A golden rule in *Wazuh* is that the version of the manager should always be equal to or higher than the agent.

In my case:

- **Manager (Kali): v4.7.3**
- **Agent (Ubuntu): v4.14.4 (installed)**

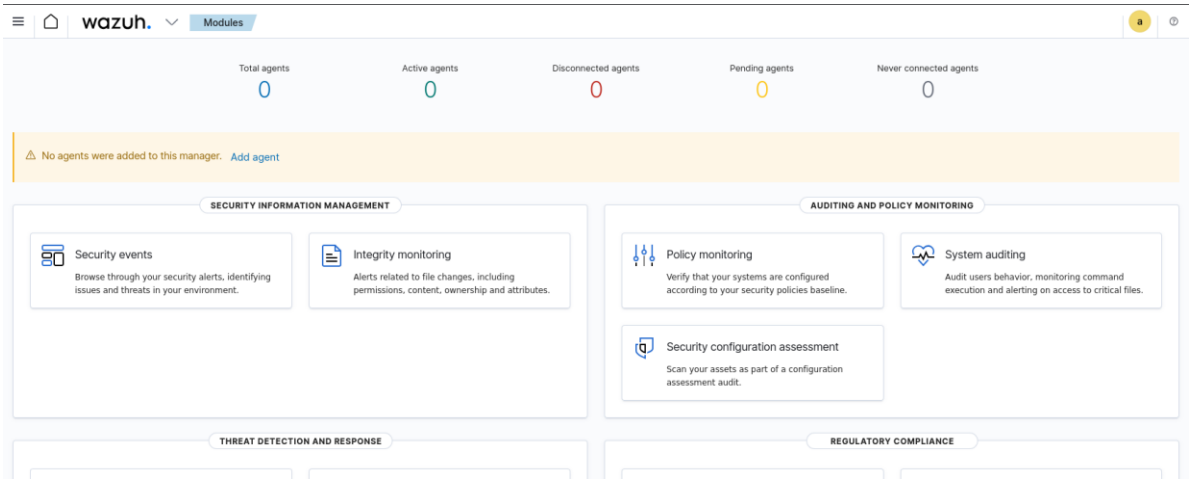
Problem:

Since the version of the agent is much newer than the manager, the data sent from the agent cannot be decoded by the manager properly. This may result in the agent showing as *'Never Connected'* or *'Disconnected'* in the dashboard.



3. Troubleshooting and Firewall Configuration

When the dashboard shows *Active Agent: 0,* there are usually two main reasons: *Network Blocking* or *Configuration/Version Mismatch*.



a. Network Verification and Port Opening

I worked on the network layer to ensure communication between the manager and the agent.

- **Ping Test:**

The following command was used to verify that the physical network connectivity is *OK*.

```
osboxes@hasib:~$ ping -c 3 192.168.91.132
PING 192.168.91.132 (192.168.91.132) 56(84) bytes of data.
64 bytes from 192.168.91.132: icmp_seq=1 ttl=64 time=1.81 ms
64 bytes from 192.168.91.132: icmp_seq=2 ttl=64 time=0.458 ms
64 bytes from 192.168.91.132: icmp_seq=3 ttl=64 time=0.429 ms

--- 192.168.91.132 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2043ms
rtt min/avg/max/mdev = 0.429/0.897/1.806/0.642 ms
```

- **Port Configuration:**

Wazuh uses ports *1514* (*Events*) and *1515* (*Enrollment*) to communicate with the agent manager. Opening these ports in *Ubuntu's* built-in firewall (*UFW*) has eliminated the traffic jamming issue.

```
osboxes@hasib:~$ sudo ufw allow 1514/tcp
sudo ufw allow 1515/tcp
Rules updated
Rules updated (v6)
Rules updated
Rules updated (v6)
```

b. Root Cause Identification

Since the network was fine but I couldn't connect, I took help from the logs, which showed me the real problem.

Command:

sudo grep -iE "error|warning" /var/ossec/logs/ossec.log

```
osboxes@hasib:~$ sudo grep -iE "error|warning" /var/ossec/logs/ossec.log | tail -n 10
2026/04/13 09:38:55 wazuh-agentd: ERROR: Agent version must be lower or equal to manager version (from manager)
2026/04/13 09:38:55 wazuh-agentd: ERROR: Unable to add agent (from manager)
2026/04/13 09:39:20 wazuh-agentd: ERROR: Agent version must be lower or equal to manager version (from manager)
2026/04/13 09:39:20 wazuh-agentd: ERROR: Unable to add agent (from manager)
2026/04/13 09:39:50 wazuh-agentd: ERROR: Agent version must be lower or equal to manager version (from manager)
2026/04/13 09:39:50 wazuh-agentd: ERROR: Unable to add agent (from manager)
2026/04/13 09:40:25 wazuh-agentd: ERROR: Agent version must be lower or equal to manager version (from manager)
2026/04/13 09:40:25 wazuh-agentd: ERROR: Unable to add agent (from manager)
2026/04/13 09:41:05 wazuh-agentd: ERROR: Agent version must be lower or equal to manager version (from manager)
2026/04/13 09:41:05 wazuh-agentd: ERROR: Unable to add agent (from manager)
```

The output shows that > *ERROR: Agent* version must be lower or equal to manager version.

This line in my log is the "*Smoking Gun*". It confirmed that the agent version (*4.14.4*) was higher than the manager version *(4.7.3*), which is not acceptable in the *Wazuh* architecture.

4. Troubleshooting: Rollback to the correct version (v4.7.2)

I removed the wrong version and installed a version compatible with the manager.

Clean uninstall

Command:

sudo apt remove wazuh-agent -y

```
osboxes@hasib:~$ sudo apt remove wazuh-agent -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages will be REMOVED:
  wazuh-agent
0 upgraded, 0 newly installed, 1 to remove and 619 not upgraded.
After this operation, 48.9 MB disk space will be freed.
(Reading database ... 149708 files and directories currently instal
led.)
Removing wazuh-agent (4.14.4-1) ...
```

This removed the wrong version of binary files from the system.

Specific version installation and fix

Command:

sudo apt install wazuh-agent=4.7.2-1

```
osboxes@hasib:~$ sudo apt install wazuh-agent=4.7.2-1
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  wazuh-agent
0 upgraded, 1 newly installed, 0 to remove and 619 not upgraded.
Need to get 9,379 kB of archives.
After this operation, 31.5 MB of additional disk space will be used.
Get:1 https://packages.wazuh.com/4.x/apt stable/main amd64 wazuh-agent amd64 4.7.2-1 [9,379 kB]
Fetched 9,379 kB in 1s (8,404 kB/s)
Preconfiguring packages ...
Selecting previously unselected package wazuh-agent.
(Reading database ... 149268 files and directories currently instal
led.)
Preparing to unpack .../wazuh-agent_4.7.2-1_amd64.deb ...
Unpacking wazuh-agent (4.7.2-1) ...
Setting up wazuh-agent (4.7.2-1) ...

Configuration file '/etc/systemd/system/wazuh-agent.service'
 ==> Deleted (by you or by a script) since installation.
 ==> Package distributor has shipped an updated version.
   What would you like to do about it ?  Your options are:
    Y or I  : install the package maintainer's version
```

Many times after downgrading, the service may show Failed due to old configuration files.

```
osboxes@hasib:~$ sudo systemctl restart wazuh-agent
Job for wazuh-agent.service failed because the control process exited with error code.
See "systemctl status wazuh-agent.service" and "journalctl -xeu wazuh-agent.service" for details.
```

I then used the --*reinstall flag* to cleanly write the files, which is a nice troubleshooting move.

```
osboxes@hasib:~$ sudo apt install --reinstall wazuh-agent=4.7.2-1
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
0 upgraded, 0 newly installed, 1 reinstalled, 0 to remove and 619 not upgraded.
Need to get 9,379 kB of archives.
After this operation, 0 B of additional disk space will be used.
Get:1 https://packages.wazuh.com/4.x/apt stable/main amd64 wazuh-agent amd64 4.7.2-1 [9,379 kB]
Fetched 9,379 kB in 1s (7,469 kB/s)
Preconfiguring packages ...
(Reading database ... 149631 files and directories currently instal
led.)
Preparing to unpack .../wazuh-agent_4.7.2-1_amd64.deb ...
Unpacking wazuh-agent (4.7.2-1) over (4.7.2-1) ...
Setting up wazuh-agent (4.7.2-1) ...
```

5. Configuration Update and Agent Activation

I manually edited the configuration file and introduced the agent to the manager.

1. Edit the configuration file (ossec.conf)

This file works like the brain of the *Wazuh* agent. Here, if the manager *IP* is wrong or if the *MANAGER_IP* remains as a variable, the connection is not established.

The config file was edited earlier with the *Nano* editor.

```
osboxes@hasib:~$ sudo nano /var/ossec/etc/ossec.conf
```

I changed *MANAGER_IP* inside the *<client><server><address>* tag of the file to *Kali*'s *IP 192.168.91.132*.

```
<ossec_config>
  <client>
    <server>
      <address>192.168.91.132</address>
      <port>1514</port>
      <protocol>tcp</protocol>
    </server>
    <config-profile>ubuntu, ubuntu24, ubuntu24.04</config-profile>
    <notify_time>10</notify_time>
    <time-reconnect>60</time-reconnect>
    <auto_restart>yes</auto_restart>
    <crypto_method>aes</crypto_method>
  </client>
```

This is important because whenever the agent starts, it first tries to find this address and perform a secure handshake on it.

2. Manual Service Start (The Hard Way)

When *systemctl* does not respond for some reason, the safest way is to use *Wazuh*'s own control script.

Command:

```
osboxes@hasib:~$ sudo /var/ossec/bin/wazuh-control start
```

The output shows that it directly runs the *wazuh-modulesd* and *wazuh-agentd* processes in memory. As a result, the agent starts working directly without background services.

```
osboxes@hasib:~$ sudo /var/ossec/bin/wazuh-control start
Starting Wazuh v4.7.2...
Started wazuh-execd...
Started wazuh-agentd...
Started wazuh-syscheckd...
Started wazuh-logcollector...
Started wazuh-modulesd...
Completed.
```

Checking Ubuntu's own IP:

```
osboxes@hasib:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:0c:29:11:4a:da brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.91.134/24 brd 192.168.91.255 scope global dynamic noprefixroute ens33
       valid_lft 1714sec preferred_lft 1714sec
    inet6 fe80::f6a9:64ea:168b:40bb/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Here, I see that *Ubuntu*'s *IP* is *192.168.91.134*.

Infrastructure Overview

After checking the IP address, the lab image now looks like this:

Machine	IP Address	Role
Kali Linux	192.168.91.132	Manager (SIEM Server)
Ubuntu VM	192.168.91.134	Agent (End-point)

Windows Agent Configuration

I have extended the scope of the project by adding a *Windows* endpoint. Here, I have completed the automated installation using *PowerShell*.

Agent Download:

Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.7.3-1.msi -OutFile wazuh-agent.msi

```
mprovements! https://aka.ms/PSWindows
PS C:\Windows\system32> Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.7.2-1.msi -OutFile wazuh-agent.msi
```

This downloads the *Wazuh installer (.msi file)* directly from the repository through Windows.

Silent installation:

msiexec.exe /i wazuh-agent.msi /q WAZUH_MANAGER="192.168.91.132" WAZUH_AGENT_GROUP="default"

```
PS C:\Windows\system32> msiexec.exe /i wazuh-agent.msi /q WAZUH_MANAGER="192.168.91.132" WAZUH_REGISTRATION_SERVER="192.168.91.132" WAZUH_AGENT_GROUP="default"
```

msiexec is the Windows installer engine. */q* stands for *'Quiet'* or silent installation, which completes the task in the background without any pop-ups. Here the agent is identified with the server by the manager's *IP*.

Starting the service:

Command:

```
PS C:\Windows\system32> NET START WazuhSvc
```

Output:

```
PS C:\Windows\system32> NET START WazuhSvc
The Wazuh service is starting.
The Wazuh service was started successfully.
```

Output

The *Wazuh* service was started successfully means that *Windows* is now connected to the dashboard.

Creating Custom OSSEC Rules and Logic Analysis

This is the most important "*intelligent*" part. The default rules often miss many attacks, so I created new rules with custom logic.

Entering and editing containers:

Commands:

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ docker exec -it single-node-wazuh.manager-1 bash
```

This is taken directly from the *Kali Linux (Server)* main terminal into the *Wazuh Manager* container so you can edit the configuration file.

Rule ID and Logic Analysis:

Rule ID	Level (Severity)	Logic and Description (Severity)	Why is this important?
100001	**12 (High)**	5 login failures in 60 seconds	This is the main rule for detecting SSH Brute Force attacks. If you enter the wrong password 5 times, a high alert will be generated.
100003	**14 (Critical)**	ROOT LOGIN attempt	Trying to log in as the root user is a big risk. Level 14 means it will give a red alert on the dashboard.
100004	**8 (Medium)**	New user account created	It will track whether hackers are entering and creating new users according to the MITRE T1136 technique.

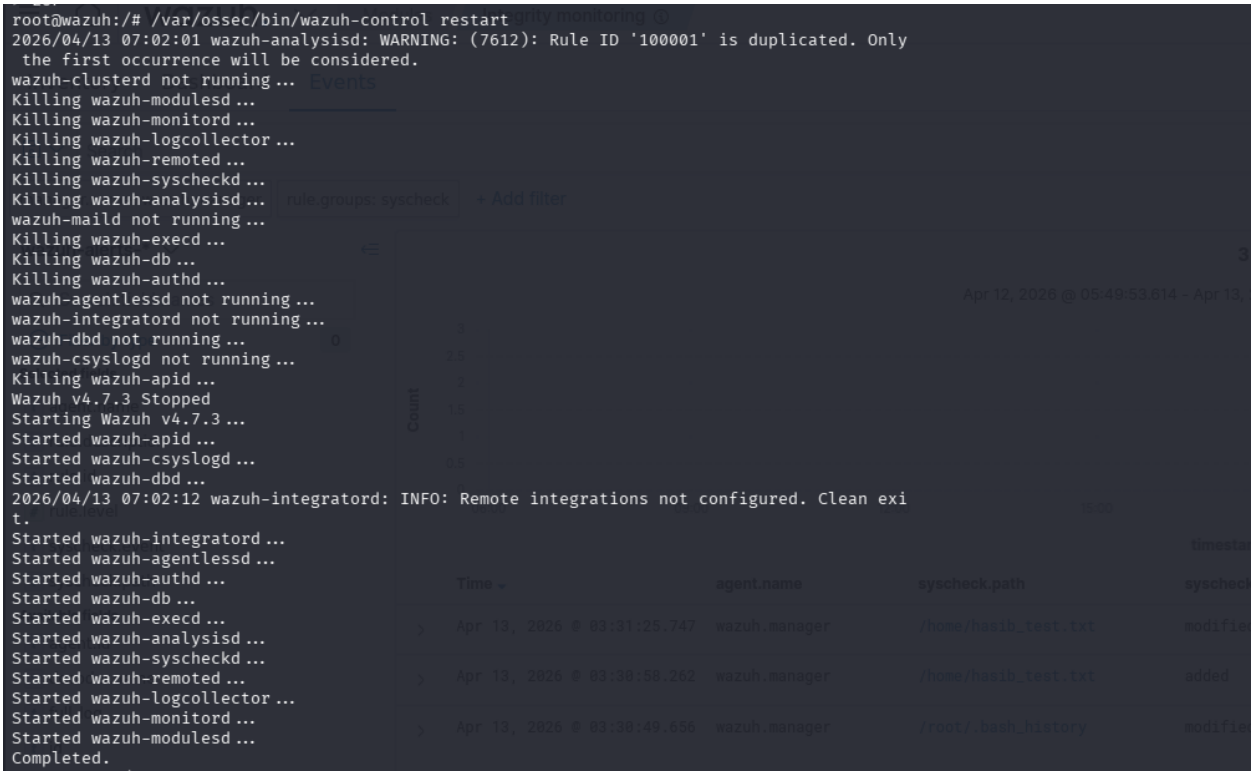
Configuration Implementation and Service Restart

The rules are injected into the *local_rules.xml* file using the *cat >> ...* command inside the container.

```
root@wazuh:/# cat >> /var/ossec/etc/rules/local_rules.xml << 'EOF'
> <group name="local,custom,">
>
>   <rule id="100001" level="12" frequency="5" timeframe="60">
>     <if_matched_sid>5760</if_matched_sid>
>     <description>SSH Brute Force Attack Detected!</description>
>     <group>authentication_failures,brute_force,</group>
>     <mitre>
>       <id>T1110</id>
>       <id>T1110.001</id>
>     </mitre>
>   </rule>
>
>   <rule id="100002" level="8">
>     <if_sid>5503</if_sid>
>     <match>Failed password</match>
>     <description>Multiple SSH Login Failures</description>
>     <group>authentication_failures,</group>
>   </rule>
>
>   <rule id="100003" level="14">
>     <if_sid>5501</if_sid>
>     <match>ROOT LOGIN</match>
>     <description>Root SSH login attempt - CRITICAL!</description>
>     <group>authentication_failures,rootcheck,</group>
>   </rule>
>
> </group>
> EOF
```

Command:

/var/ossec/bin/wazuh-control restart



It is mandatory to restart the manager after making any changes to the rules file. This compiles the new *XML* rules and loads them into memory.

Exit

Then I Exit the container and return to the main Kali system.



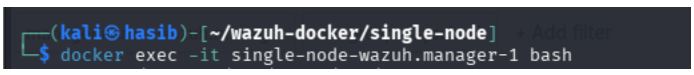
File Integrity Monitoring (FIM) Configuration

The *FIM* module basically monitors the *Checksum* (*MD5, SHA1, SHA256*) of the file. Whenever the content or permissions of the file are changed, *Wazuh* generates an alert.

Entering and Editing the Manager Container

First, the configuration file is opened by entering the *Wazuh Manager* container in *Kali.*

Entering the Container



Open ossec.conf in nano editor

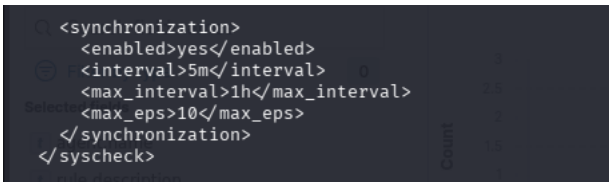
```
root@wazuh:/# nano /var/ossec/etc/ossec.conf
```

Configuration File Update (ossec.conf)

The parameters I added in the <*syscheck*> section are very professional and are used at production level:

- **Realtime Monitoring:** Using <*directories realtime="yes"*> will alert as soon as I save the file.
- **Report Changes:** The *report_changes="yes"* parameter will also show you exactly what content has been added or removed from the file in the differential view on the dashboard.
- **Monitoring Directories:** */etc, /bin, /sbin, /home* and */root* are monitored, which are the most sensitive areas of a *Linux* system.



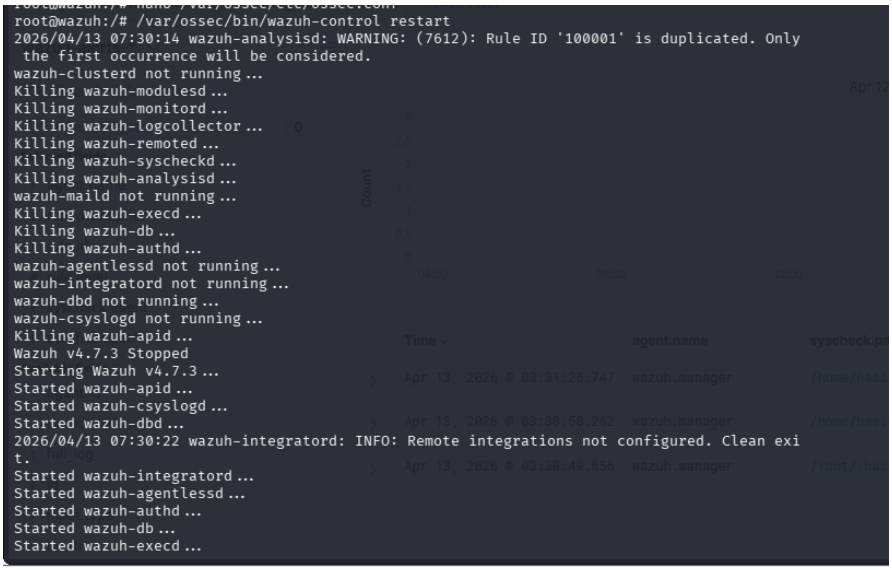


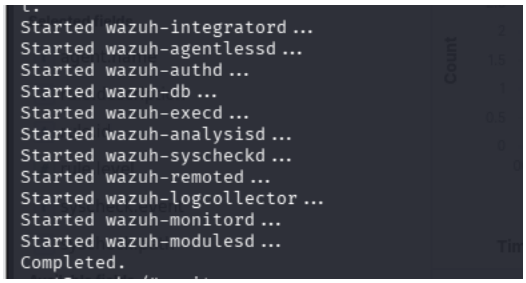
Service Restart and Output Analysis

After saving the configuration, the service was restarted using the *Wazuh* control tool.

Manager Service Restart

/var/ossec/bin/wazuh-control restart



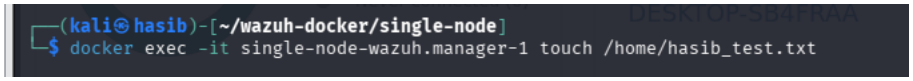


The output shows a warning: *Rule ID '7612'* is duplicated. This means that the ID of the previously created custom rule is the same as another rule. Although the service has been started, it is better to keep it unique in the future. Then I exit.



Since the manager (*Kali*) is set up inside the container, a file is created inside *Kali* and tested:

Create a test file by entering the container again



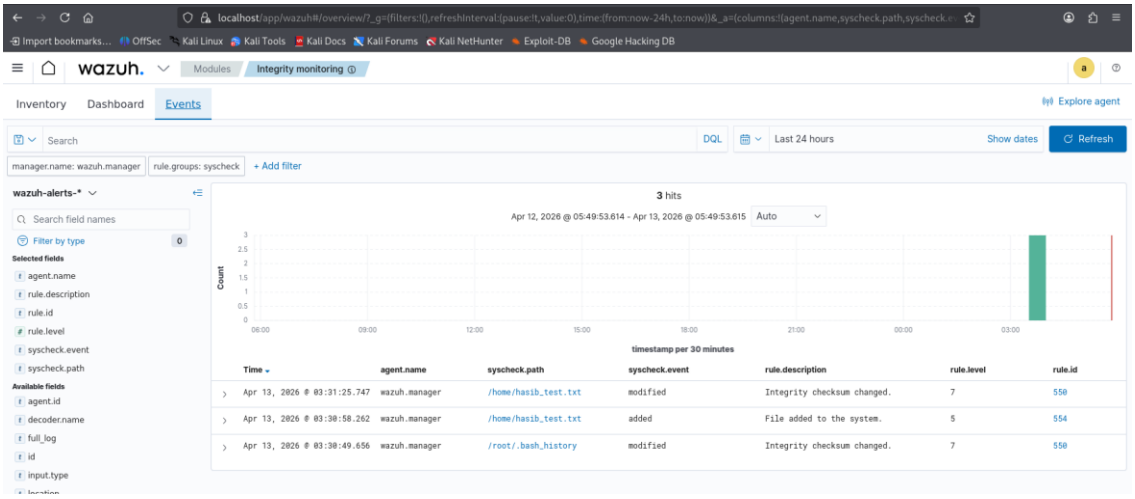
Lab Test and Real-Time Event Analysis

The test I performed with the touch */home/hasib_test.txt* command resulted in 3 different events being observed in the dashboard:

Then open the *Wazuh Dashboard* in the browser and go to Integrity Monitoring from the left-hand menu and click on the Events tab.

There I can see an alert that says:

- **Path:** */home/hasib_test.txt*
- **Event:** *File added to the system*



FIM Events Analysis

The log clearly shows that a total of 3 real-time events were detected in the *wazuh.manager* agent (i.e., the Kali container):

Event 1: Terminal History Update

- **Time:** 03:30:49
- **Path:** */root/.bash_history*
- **Action:** *Modified (Integrity checksum changed)*
- **Analysis:** A command executed in the terminal as the root user caused a change in the system's history file. *Wazuh* successfully detected this metadata change (*Rule 550, Level 7*). This proves that the */root* directory is under surveillance.

Event 2: New File Creation

- **Time:** 03:30:58
- **Path:** */home/hasib_test.txt*
- **Action:** *Added (File added to the system)*
- **Analysis:** A new file was created in the system due to the command *touch /home/hasib_test.txt* being executed in the terminal. This alert is triggered in real-time as soon as the file is created (*Rule 554, Level 5*).

Event 3: File modification or metadata update

- **Time:** 03:31:25
- **Path:** */home/hasib_test.txt*
- **Action:** *Modified (Integrity checksum changed)*
- **Analysis:** The file was modified (e.g. edited or permissions changed) shortly after it was created, which caused the file's hash value (*Checksum*) to change. *Wazuh* was able to detect this modification very quickly (*Rule 550, Level 7*).

Task 3 - Play with SIEM in action

Wazuh provides a numerous possibilities for Security Operations Center (SOC) implementation such as:

- **Log Data Analysis**
- **Cloud Security**
- **Container Security**
- **File Integrity Monitoring**
- **Intrusion Detection**
- **Security Audit and Analytics**
- **Vulnerability Detection**
- **Policy Auditing**
- **Monitoring Logons**
- **Active Response**
- **Incident Response**
- **...**

By the way, we suggest you the default and most popular cases of SIEM usage that you might take:

- **Active Response**
- **Brute-force attack simulation with attacker user blocking**
- **Malware detection by SIEM**

By simulating attacks and security threats or exploiting vulnerabilities on agents, try out as many of the above Wazuh features in action as possible. The number of features is limited only by your capabilities, resources and time. Nevertheless, try to test at least two different Wazuh features.

Regarding how to check the operability of your SIEM system and get security incidents. In fact, it is absolutely your freedom to choose which methods and techniques to use for this. First of all, they are related to the functionality of Wazuh that you plan to test. Consequently, the choice of methods and tools for emulating security threats on agents is really wide, so here we just give a some motley sketch of examples:

- **testmynids**
- **hydra brute force**
- **Atomic Red Team tests library**
- **Caldera**
- **DVWA**
- **Nikto web scanner**
- **Detection guides on Wazuh official blog**
- **...**

At the end, also learn Wazuh Incident Response features, generate Wazuh security reports and analyze its content.

Solution:

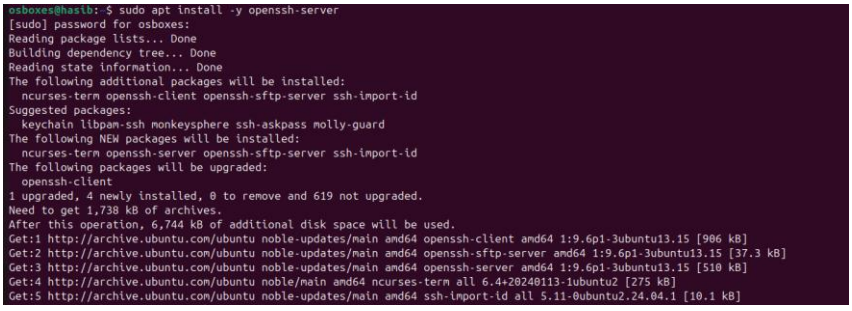
Brute Force Attack Simulation (Hydra)

Enabling SSH on Ubuntu Agent (on Ubuntu VM):

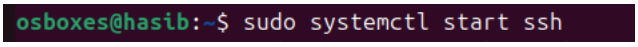
Installing openssh-server:

Command:

sudo apt install -y openssh-server



Starting ssh:



Enabling:

Command:

sudo systemctl enable ssh

```
osboxes@hasib:~$ sudo systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink /etc/systemd/system/sshd.service → /usr/lib/systemd/system/ssh.service.
Created symlink /etc/systemd/system/multi-user.target.wants/ssh.service → /usr/lib/systemd/system/ssh.service.
```

Checking Status:

Command:

sudo systemctl status ssh

```
osboxes@hasib:~$ sudo systemctl status ssh
○ ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: inactive (dead)
TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
   Main PID: 18994 (sshd)
      Tasks: 1 (limit: 4615)
     Memory: 1.2M (peak: 1.4M)
        CPU: 26ms
     CGroup: /system.slice/ssh.service
             └─18994 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Apr 13 15:13:03 hasib systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Apr 13 15:13:03 hasib sshd[18994]: Server listening on 0.0.0.0 port 22.
Apr 13 15:13:03 hasib sshd[18994]: Server listening on :: port 22.
Apr 13 15:13:03 hasib systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
```

The output shows "*Active: active (running)*".

Knowing the IP of the Ubuntu VM:

```
osboxes@hasib:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:0c:29:11:4a:da brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.91.134/24 brd 192.168.91.255 scope global dynamic noprefixroute ens33
       valid_lft 1137sec preferred_lft 1137sec
    inet6 fe80::f6a9:64ea:168b:40bb/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

The IP of *ens33* is *192.168.91.134*.

Creating a Password List in Kali Linux:

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ cat > /tmp/passwords.txt << 'EOF'
password
345756
admin
ubuntu
root
test347
welcome
password876
qwerty
letmein
changeme
ubuntu784
EOF
```

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ echo "ubuntu" > /tmp/users.txt
```

Creating a list of usernames

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ echo -e "admin\nroot\nubuntu\nosboxes\nhasib" > /tmp/users.txt
```

Creating a password list

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ echo -e "12345\npassword\nadmin123\n123456\nqwerty" > /tmp/passwords.txt
```

Performing Brute Force Attack with Hydra

Setting the correct IP of Ubuntu Agent

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ export AGENT_IP="192.168.91.134"
```

Starting Hydra attack:

Command:

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ hydra -L /tmp/users.txt -P /tmp/passwords.txt ssh://$AGENT_IP -t 4 -v -f
```

output:

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ hydra -L /tmp/users.txt -P /tmp/passwords.txt ssh://$AGENT_IP -t 4 -v -f
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

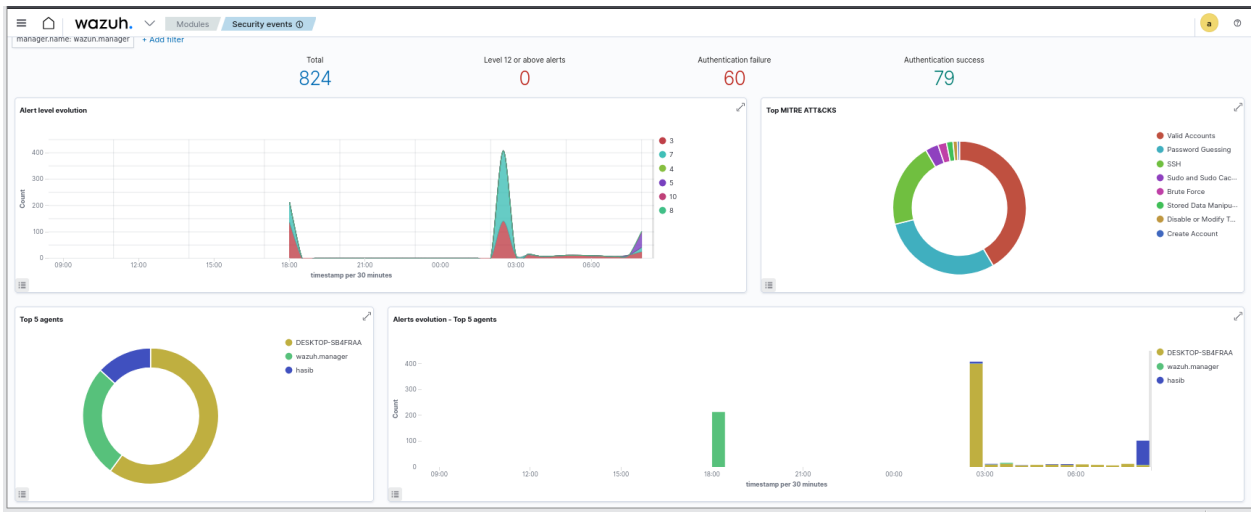
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-13 08:22:07
[DATA] max 4 tasks per 1 server, overall 4 tasks, 25 login tries (l:5/p:5), ~7 tries per task
[DATA] attacking ssh://192.168.91.134:22/
[VERBOSE] Resolving addresses ... [VERBOSE] resolving done
[INFO] Testing if password authentication is supported by ssh://admin@192.168.91.134:22
[INFO] Successful, password authentication is supported by ssh://192.168.91.134:22
[STATUS] attack finished for 192.168.91.134 (waiting for children to complete tests)
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-13 08:22:25
```

The flags mean:

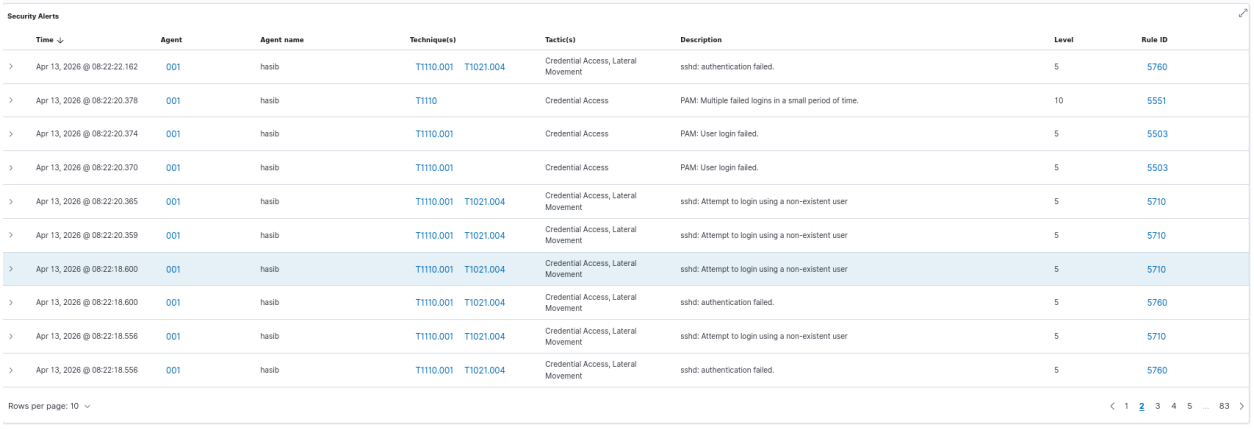
- -L = username list file
- -P = password list file
- -t 4 = 4 parallel connections
- -v = verbose
- -f = Stop when you get the first match

View Alert on Dashboard:

I went to *Wazuh Dashboard* and clicked on *Security Events* from the left menu and select *Last 30 minutes* from the *Time picker* at the top and typeed *rule.groups: authentication_failures* in the *Search bar*. Then I clicked *Enter*. In the Alert list, I saw that *Attacker IP, Target user*, *Timestamp, Rule ID* are all there.



I go to the *Threat Hunting* menu and enter *data.srcip: 192.168.91.132*, all events according to the attacker IP are be displayed.



Security Alerts

Time	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
Apr 13, 2026 @ 08:22:22.162	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: authentication failed.	5	5760
Apr 13, 2026 @ 08:22:20.378	001	hasib	T1110	Credential Access	PAM: Multiple failed logins in a small period of time.	10	5551
Apr 13, 2026 @ 08:22:20.374	001	hasib	T1110.001	Credential Access	PAM: User login failed.	5	5503
Apr 13, 2026 @ 08:22:20.370	001	hasib	T1110.001	Credential Access	PAM: User login failed.	5	5503
Apr 13, 2026 @ 08:22:20.365	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
Apr 13, 2026 @ 08:22:20.359	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
Apr 13, 2026 @ 08:22:18.600	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
Apr 13, 2026 @ 08:22:18.600	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: authentication failed.	5	5760
Apr 13, 2026 @ 08:22:18.556	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
Apr 13, 2026 @ 08:22:18.556	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: authentication failed.	5	5760

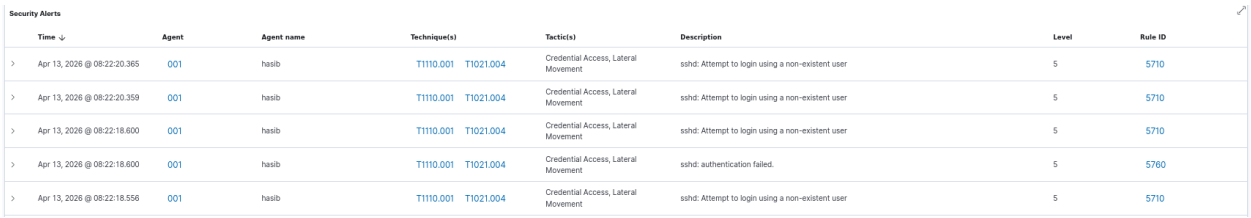
Rows per page: 10

›	Apr 13, 2026 @ 08:22:22.162	001	hasib	T1110	Credential Access	sshd: brute force trying to get access to the system. Authentication failed.	10	5763

1. Security Events Analysis

If I go to the Threat Hunting menu and enter *data.srcip: 192.168.91.132*, you can see all the events according to the attacker IP. As I can see from the dashboard, *Wazuh* generated several types of alerts as soon as the attack started:

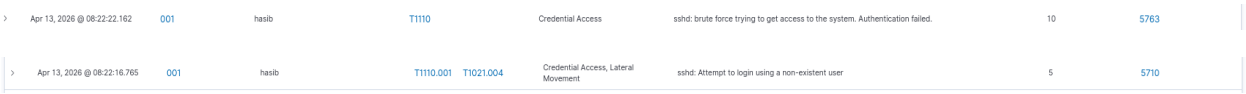
- **Rule 5763 (Level 10):** This is my biggest success. *Wazuh* has detected that this is a Brute Force Attack. The description reads: *"sshd: brute force trying to get access to the system".*
- **Rule 5551 (Level 10):** In my second screenshot, I can see that *PAM (Pluggable Authentication Modules)* has detected that there have been many failed login attempts in a very short period of time.
- **Rule 5710 (Level 5):** When *Hydra* was trying with admin or any other user that does not exist on my *Ubuntu system*, this alert came: *"sshd: Attempt to login using a non-existent user".*



Security Alerts

	Time ↓	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
›	Apr 13, 2026 @ 08:22:20.365	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
›	Apr 13, 2026 @ 08:22:20.359	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
›	Apr 13, 2026 @ 08:22:18.600	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
›	Apr 13, 2026 @ 08:22:18.600	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: authentication failed.	5	5760
›	Apr 13, 2026 @ 08:22:18.556	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710

- **Rule 5760 & 5503 (Level 5):** These are general password error alerts that are generated for each Hydra attempt.



›	Apr 13, 2026 @ 08:22:22.162	001	hasib	T1110	Credential Access	sshd: brute force trying to get access to the system. Authentication failed.	10	5763
›	Apr 13, 2026 @ 08:22:16.765	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710

2. MITRE ATT&CK Mapping

Notice that *Wazuh* has automatically mapped these attacks to the *MITRE ATT&CK* framework in the dashboard:

• **T1110 (Brute Force):** This falls directly under the *'Credential Access'* tactic.

• **T1021.004 (SSH):** This is identified as a *'Lateral Movement'* tactic.

Successfully simulated *SSH Brute Force* attack on *Ubuntu Agent* using *Hydra from Kali Linux.*

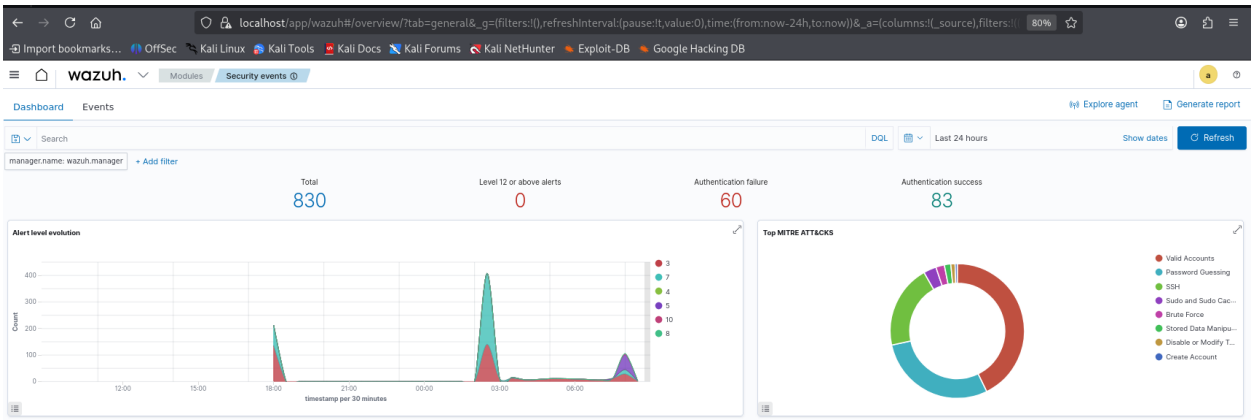
Wazuh Dashboard

Also, ***Rule ID 5763 (Level 10)*** is triggered in real-time in the dashboard, which confirms that our *SIEM* system is able to detect suspicious login activities within the network and classify them according to the *MITRE ATT&CK* framework.

Event Statistics Analysis

After the attack simulation, I got the following information from the dashboard summary:

- **Total Alerts (830):** The total number of alerts in the system is *830*. This is the sum of all security and system events that have been accumulated since the start of my lab.
- **Authentication Failure (60):** This is basically the result of my *Hydra Brute Force Attack*. This number increases rapidly as *Hydra* tries to enter the wrong password repeatedly. This proves that the system records every unauthorized attempt.
- **Authentication Success (83):** There have been *83* successful logins to the system in the past few days. This usually indicates valid logins made to my Ubuntu or Kali.
- **Level 12 or above alerts (0):** There are currently no critical threats to the system. Usually, alerts would appear here if there was a direct system compromise or major malware activity.



Active Response - Automatic IP Blocking

Active Response means that *Wazuh* itself blocks the attacker's *IP* through the firewall.

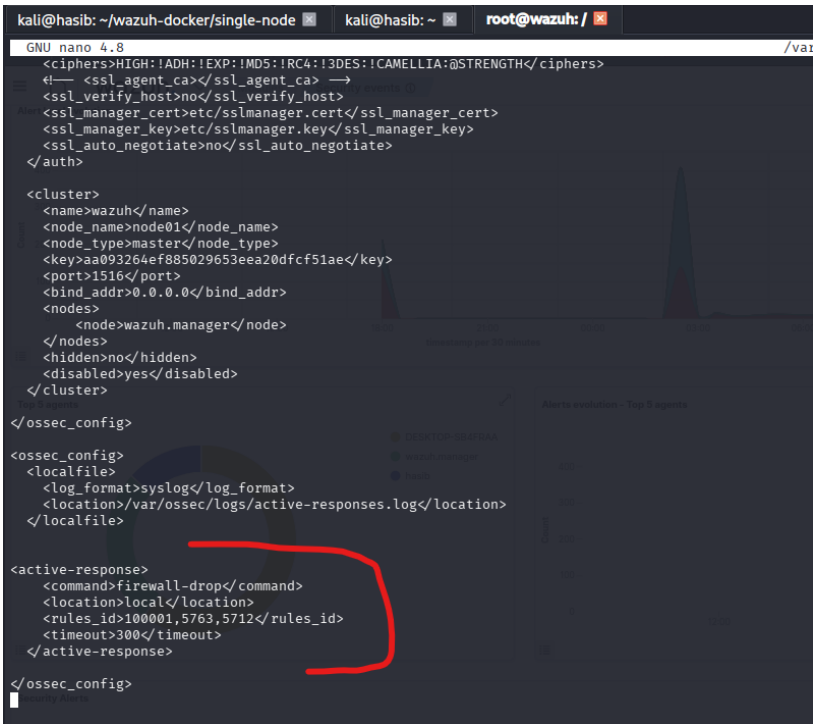
Configured in Manager Container:

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ docker exec -it single-node-wazuh.manager-1 bash
```

Setting up the config correctly:

```
root@wazuh:/# nano /var/ossec/etc/ossec.conf
```

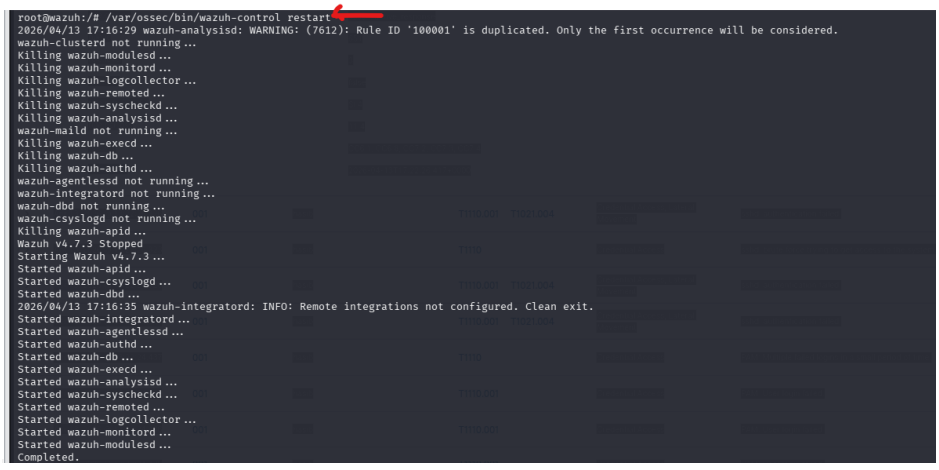
This is added just before the </ossec_config> closing tag:



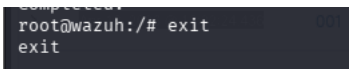
- *firewall-drop* = block IP with iptables
- *timeout>300* = automatic unblock after 300 seconds (5 minutes)

Restart:

/var/ossec/bin/wazuh-control restart



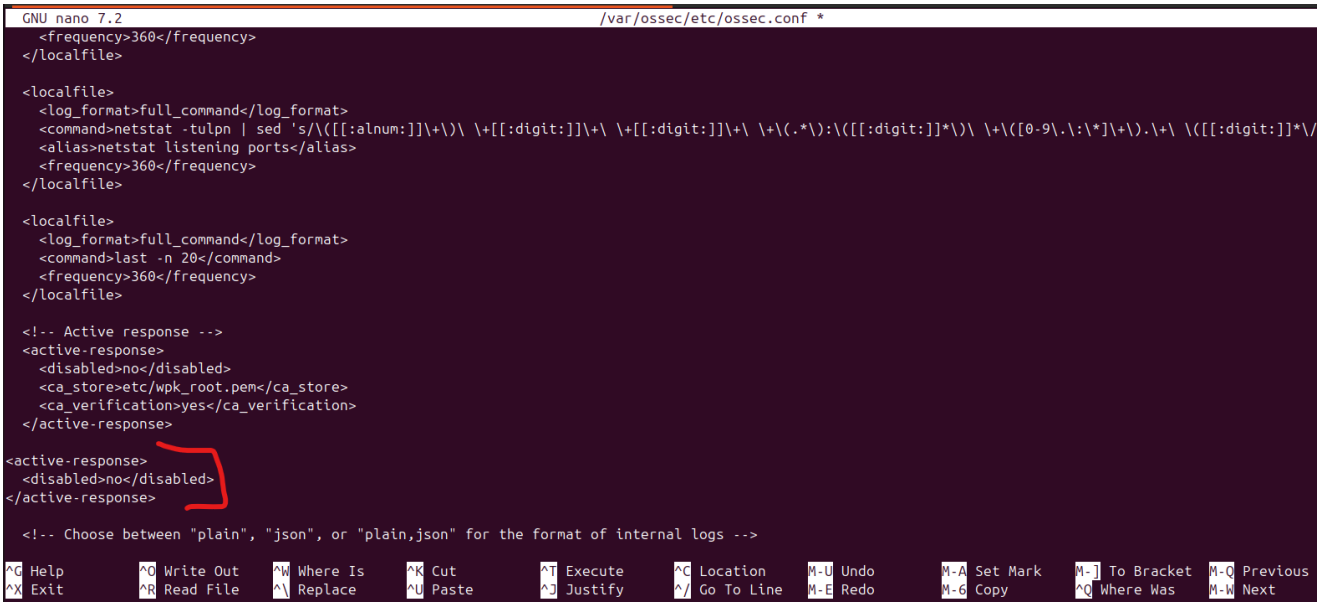
Then take Exit:



Enable Ubuntu Agent as well:

```
osboxes@hasib:~$ sudo nano /var/ossec/etc/ossec.conf
```

Added inside <ossec_config>:



Restarting the Agent:

```
osboxes@hasib:~$ sudo systemctl restart wazuh-agent
```

Increase password list:

The password list in *Wazuh* Server is made to be at least 30-40 lines. Without this, the Brute Force rule does not trigger. For this, the following command is given:

Add 50 passwords with numbers from 1 to 50:

```
osboxes@hasib:~$ for i in {1..50}; do echo "password$i" >> /tmp/passwords.txt; done
```

Running Hydra again on Wazuh Server for testing:

Fixing Hydra tasks and threads:

Then the attack was executed with the following command. Here *-t 4* (4 parallel tasks) is given so that the Ubuntu machine can "*suffer*" the attack and *Wazuh* has time to process it.

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ hydra -l osboxes -P /tmp/passwords.txt 192.168.91.134 ssh -v
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these **s ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-13 13:09:36
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 5 tasks per 1 server, overall 5 tasks, 5 login tries (l:1/p:5), ~1 try per task
[DATA] attacking ssh://192.168.91.134:22/
[VERBOSE] Resolving addresses ... [VERBOSE] resolving done
[INFO] Testing if password authentication is supported by ssh://osboxes@192.168.91.134:22
[INFO] Successful, password authentication is supported by ssh://192.168.91.134:22
[STATUS] attack finished for 192.168.91.134 (waiting for children to complete tests)
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-13 13:09:40
```

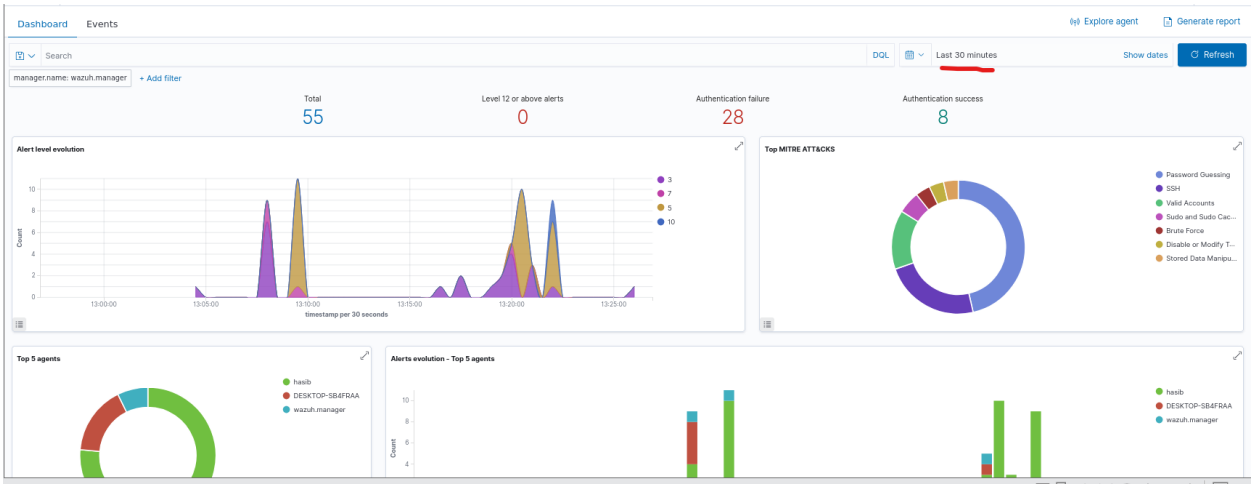
After a while, the IP of *Wazuh* Server gets blocked.

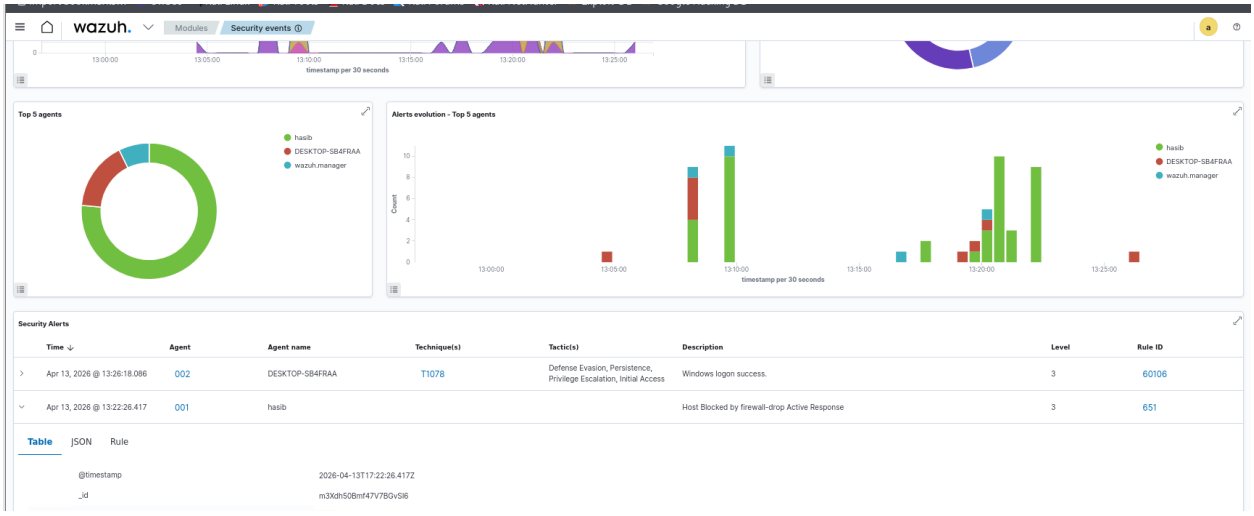
Viewing blocked IP on Ubuntu VM:

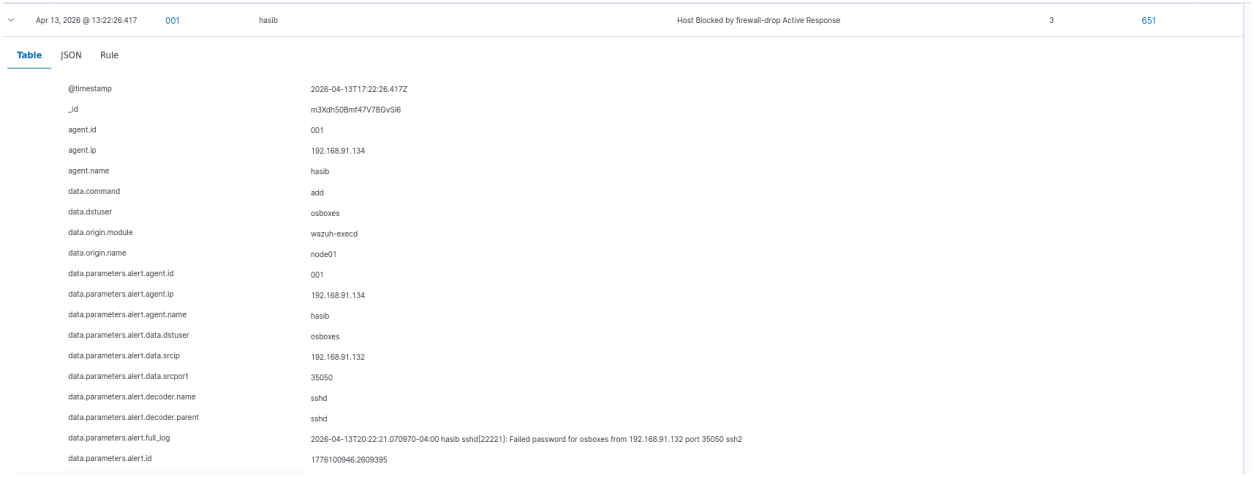
```
osboxes@hasib:~$ sudo iptables -L INPUT -n | grep DROP
DROP       0    --  192.168.91.132       0.0.0.0/0
```

The output shows that Kali's IP (*192.168.91.132*) has been successfully added to the *DROP* list.

I go to Security Events from the Dashboard and search for *active_response,* I can see the blocking event. The time given to view is last 30 minutes.



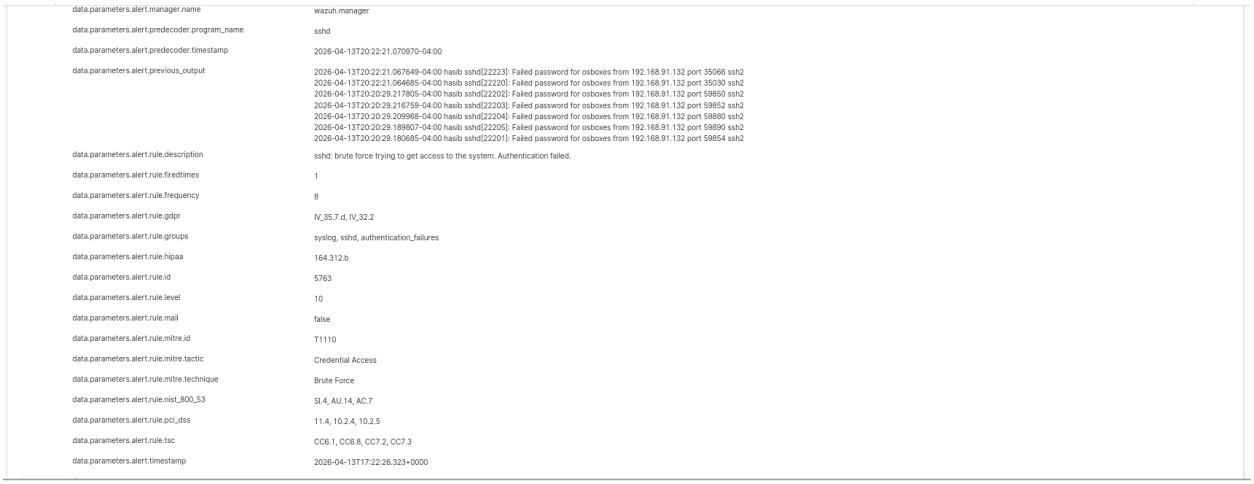




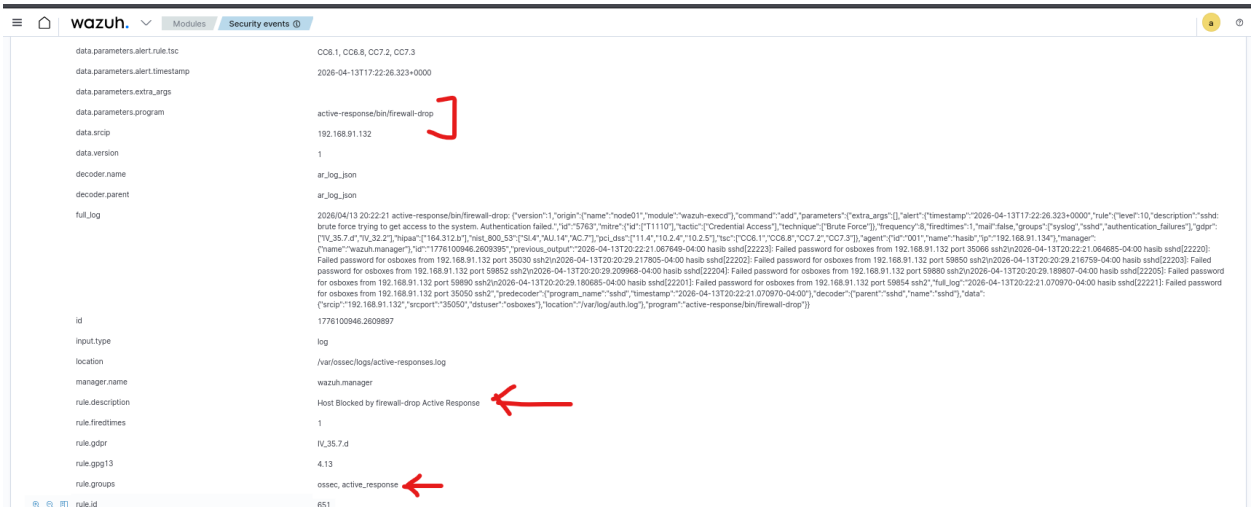
Apr 13, 2026 @ 13:22:26.417 | 001 | hasib | Host Blocked by firewall-drop Active Response | 3 | 651

Table | JSON | Rule

@timestamp	2026-04-13T17:22:26.417Z
_id	m3Xdh508mf47V7BGvSl6
agent.id	001
agent.ip	192.168.91.134
agent.name	hasib
data.command	add
data.dstuser	osboxes
data.origin.module	wazuh-execd
data.origin.name	node01
data.parameters.alert.agent.id	001
data.parameters.alert.agent.ip	192.168.91.134
data.parameters.alert.agent.name	hasib
data.parameters.alert.data.dstuser	osboxes
data.parameters.alert.data.srcip	192.168.91.132
data.parameters.alert.data.srcport	35050
data.parameters.alert.decoder.name	sshd
data.parameters.alert.decoder.parent	sshd
data.parameters.alert.full_log	2026-04-13T20:22:21.070970-04:00 hasib sshd[22221]: Failed password for osboxes from 192.168.91.132 port 35050 ssh2
data.parameters.alert.id	1776100946.2609395



data.parameters.alert.manager.name	wazuh.manager
data.parameters.alert.predecoder.program_name	sshd
data.parameters.alert.predecoder.timestamp	2026-04-13T20:22:21.070970-04:00
data.parameters.alert.previous_output	2026-04-13T20:22:21.067649-04:00 hasib sshd[22223]: Failed password for osboxes from 192.168.91.132 port 35066 ssh2 2026-04-13T20:22:21.064685-04:00 hasib sshd[22220]: Failed password for osboxes from 192.168.91.132 port 35030 ssh2 2026-04-13T20:20:29.217805-04:00 hasib sshd[22202]: Failed password for osboxes from 192.168.91.132 port 59850 ssh2 2026-04-13T20:20:29.216759-04:00 hasib sshd[22203]: Failed password for osboxes from 192.168.91.132 port 59852 ssh2 2026-04-13T20:20:29.209968-04:00 hasib sshd[22204]: Failed password for osboxes from 192.168.91.132 port 59880 ssh2 2026-04-13T20:20:29.189807-04:00 hasib sshd[22205]: Failed password for osboxes from 192.168.91.132 port 59890 ssh2 2026-04-13T20:20:29.180685-04:00 hasib sshd[22201]: Failed password for osboxes from 192.168.91.132 port 59854 ssh2
data.parameters.alert.rule.description	sshd: brute force trying to get access to the system. Authentication failed.
data.parameters.alert.rule.firedtimes	1
data.parameters.alert.rule.frequency	8
data.parameters.alert.rule.gdpr	IV_35.7.d, IV_32.2
data.parameters.alert.rule.groups	syslog, sshd, authentication_failures
data.parameters.alert.rule.hipaa	164.312.b
data.parameters.alert.rule.id	5763
data.parameters.alert.rule.level	10
data.parameters.alert.rule.mail	false
data.parameters.alert.rule.mitre.id	T1110
data.parameters.alert.rule.mitre.tactic	Credential Access
data.parameters.alert.rule.mitre.technique	Brute Force
data.parameters.alert.rule.nist_800_53	SI.4, AU.14, AC.7
data.parameters.alert.rule.pci_dss	11.4, 10.2.4, 10.2.5
data.parameters.alert.rule.tsc	CC6.1, CC6.8, CC7.2, CC7.3
data.parameters.alert.timestamp	2026-04-13T17:22:26.323+0000



wazuh. | Modules | Security events

data.parameters.alert.rule.tsc	CC6.1, CC6.8, CC7.2, CC7.3
data.parameters.alert.timestamp	2026-04-13T17:22:26.323+0000
data.parameters.extra_args	
data.parameters.program	active-response/bin/firewall-drop
data.srcip	192.168.91.132
data.version	1
decoder.name	ar_log_json
decoder.parent	ar_log_json
full_log	2026/04/13 20:22:21 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"node01","module":"wazuh-execd"},"command":"add","parameters":{"extra_args":[],"alert":{"timestamp":"2026-04-13T17:22:26.323+0000","rule":{"level":10,"description":"sshd: brute force trying to get access to the system. Authentication failed.","id":"5763","mitre":{"id":["T1110"],"tactic":["Credential Access"],"technique":["Brute Force"]},"frequency":8,"firedtimes":1,"mail":false,"groups":["syslog","sshd","authentication_failures"],"gdpr":["IV_35.7.d","IV_32.2"],"hipaa":["164.312.b"],"nist_800_53":["SI.4","AU.14","AC.7"],"pci_dss":["11.4","10.2.4","10.2.5"],"tsc":["CC6.1","CC6.8","CC7.2","CC7.3"]},"agent":{"id":"001","name":"hasib","ip":"192.168.91.134"},"manager":{"name":"wazuh.manager"},"id":"1776100946.2609395","previous_output":"2026-04-13T20:22:21.067649-04:00 hasib sshd[22223]: Failed password for osboxes from 192.168.91.132 port 35066 ssh2\n2026-04-13T20:22:21.064685-04:00 hasib sshd[22220]: Failed password for osboxes from 192.168.91.132 port 35030 ssh2\n2026-04-13T20:20:29.217805-04:00 hasib sshd[22202]: Failed password for osboxes from 192.168.91.132 port 59850 ssh2\n2026-04-13T20:20:29.216759-04:00 hasib sshd[22203]: Failed password for osboxes from 192.168.91.132 port 59852 ssh2\n2026-04-13T20:20:29.209968-04:00 hasib sshd[22204]: Failed password for osboxes from 192.168.91.132 port 59880 ssh2\n2026-04-13T20:20:29.189807-04:00 hasib sshd[22205]: Failed password for osboxes from 192.168.91.132 port 59890 ssh2\n2026-04-13T20:20:29.180685-04:00 hasib sshd[22201]: Failed password for osboxes from 192.168.91.132 port 59854 ssh2","full_log":"2026-04-13T20:22:21.070970-04:00 hasib sshd[22221]: Failed password for osboxes from 192.168.91.132 port 35050 ssh2","predecoder":{"program_name":"sshd","timestamp":"2026-04-13T20:22:21.070970-04:00"},"decoder":{"parent":"sshd","name":"sshd"},"data":{"srcip":"192.168.91.132","srcport":"35050","dstuser":"osboxes"},"location":"/var/log/auth.log"},"program":"active-response/bin/firewall-drop"}}
id	1776100946.2609897
input.type	log
location	/var/ossec/logs/active-responses.log
manager.name	wazuh.manager
rule.description	Host Blocked by firewall-drop Active Response
rule.firedtimes	1
rule.gdpr	IV_35.7.d
rule.gpg13	4.13
rule.groups	ossec, active_response
rule.id	651

Apr 13, 2026 @ 13:22:26.326	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: authentication failed.	5	5760
Apr 13, 2026 @ 13:22:26.323	001	hasib	T1110	Credential Access	sshd: brute force trying to get access to the system. Authentication failed.	10	5763
Apr 13, 2026 @ 13:22:26.320	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: authentication failed.	5	5760
Apr 13, 2026 @ 13:22:26.317	001	hasib	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: authentication failed.	5	5760
Apr 13, 2026 @ 13:22:24.437	001	hasib	T1110	Credential Access	PAM: Multiple failed logins in a small period of time.	10	5551
Apr 13, 2026 @ 13:22:24.437	001	hasib	T1110.001	Credential Access	PAM: User login failed.	5	5503
Apr 13, 2026 @ 13:22:24.436	001	hasib	T1110.001	Credential Access	PAM: User login failed.	5	5503
Apr 13, 2026 @ 13:22:24.436	001	hasib	T1110.001	Credential Access	PAM: User login failed.	5	5503

By analyzing the security alerts in the *Wazuh* dashboard, I was able to confirm three important phases of the project: attack detection, automatic response, and administrative audit.

1. Attack Detection & Response

This lab simulation showed that *Wazuh* took the following steps as soon as the attack began:

- **Brute Force Detection (Rule ID: 5763):** *Wazuh Manager* successfully detected that this was a brute force attack. According to the screenshot of the log, its description is: *"sshd: brute force trying to get access to the system. Authentication failed."* This is a *Level 10 (High)* alert that confirms that the system has detected the type of brute force attack.

Apr 13, 2026 @ 13:22:26.323	001	hasib	T1110	Credential Access	sshd: brute force trying to get access to the system. Authentication failed.	10	5763

- **Active Response Trigger (Rule ID: 651):** This is the main milestone of this lab. Wazoo recorded in the log: "*Host Blocked by firewall-drop Active Response*". This means that the system did not just sit by and watch the attack, but also took action.

Apr 13, 2026 @ 13:22:26.417	001	hasib	Host Blocked by firewall-drop Active Response	3	651

- **Attacker IP Identification:** *Wazuh* accurately identified the attacker's *Kali IP 192.168.91.132* and blocked it via the *firewall-drop* script.

data.parameters.program	active-response/bin/firewall-drop
data.srcip	192.168.91.132
data.version	1
decoder.name	ar_log_json
decoder.parent	ar_log_json
full_log	2026/04/13 20:22:21 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"node01","module":"wazuh-execd"},"command":"add","parameters":{"extra_args":[],"alert":{"timestamp":"2026-04-13T17:22:26.323+0000","rule":{"level":10,"description":"sshd: brute force trying to get access to the system. Authentication failed.","id":"5763","mitre":{"id":["T1110"],"tactic":["Credential Access"],"technique":["Brute Force"]},"frequency":8,"firedtimes":1,"mail":false,"groups":["syslog","sshd","authentication_failures"],"gdpr":["IV_35.7.d","IV_32.2"],"hipaa":["164.312.b"],"nist_800_53":["SI.4","AU.14","AC.7"],"pci_dss":["11.4","10.2.4","10.2.5"],"tsc":["CC6.1","CC6.8","CC7.2","CC7.3"]},"agent":{"id":"001","name":"hasib","ip":"192.168.91.134"},"manager":{"name":"wazuh.manager"},"id":"1776100946.2609395","previous_output":"2026-04-13T20:22:21.067649-04:00 hasib sshd[22223]: Failed password for osboxes from 192.168.91.132 port 35086 ssh2\n2026-04-13T20:22:21.064685-04:00 hasib sshd[22220]: Failed password for osboxes from 192.168.91.132 port 35030 ssh2\n2026-04-13T20:20:29.217805-04:00 hasib sshd[22202]: Failed password for osboxes from 192.168.91.132 port 59850 ssh2\n2026-04-13T20:20:29.216759-04:00 hasib sshd[22203]: Failed password for osboxes from 192.168.91.132 port 59852 ssh2\n2026-04-13T20:20:29.209968-04:00 hasib sshd[22204]: Failed password for osboxes from 192.168.91.132 port 59880 ssh2\n2026-04-13T20:20:29.189807-04:00 hasib sshd[22205]: Failed password for osboxes from 192.168.91.132 port 59890 ssh2\n2026-04-13T20:20:29.180685-04:00 hasib sshd[22201]: Failed password for osboxes from 192.168.91.132 port 59854 ssh2","full_log":"2026-04-13T20:22:21.070970-04:00 hasib sshd[22221]: Failed password for osboxes from 192.168.91.132 port 35050 ssh2","predecoder":{"program_name":"sshd","timestamp":"2026-04-13T20:22:21.070970-04:00"},"decoder":{"parent":"sshd","name":"sshd"},"data":{"srcip":"192.168.91.132","srcport":"35050","dstuser":"osboxes"},"location":"/var/log/auth.log"},"program":"active-response/bin/firewall-drop"}}
id	1776100946.2609897
input.type	log
location	/var/ossec/logs/active-responses.log
manager.name	wazuh.manager
rule.description	Host Blocked by firewall-drop Active Response
rule.firedtimes	1
rule.gdpr	IV_35.7.d
rule.gpg13	4.13
rule.groups	ossec, active_response
rule.id	651

2. Data Analysis Table

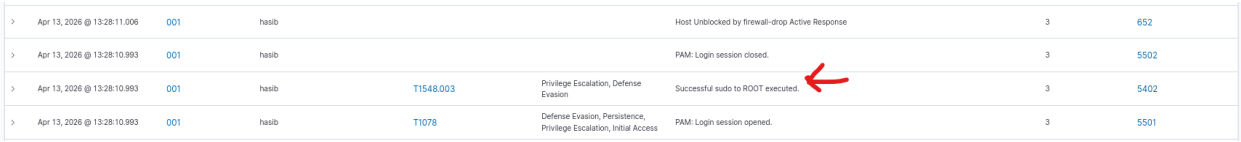
A brief summary of the logs obtained during the attack is given below:

Parameter	Value from Log	Interpretation
Alert Timestamp	Apr 13, 2026 @ 13:22:26	Specific time for attack detection and block completion.
Agent Name	hasib (192.168.91.134)	The victim (Ubuntu Agent) PC on which the attack was carried out.
Attacker IP	192.168.91.132	The attacker's IP address, which is currently blocked.
MITRE ATT&CK	T1110 (Brute Force)	Type of attack (Credential Access).
Action Taken	firewall-drop (add)	The IP is blocked by Ubuntu's firewall.
Detection Log Source	/var/log/auth.log	This information was obtained by analyzing the system file.

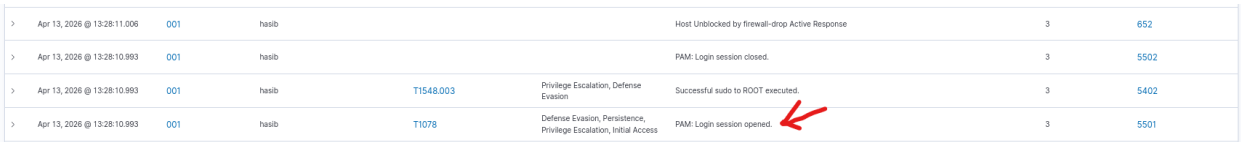
Administrative Activity Monitoring

When the system administrator starts the investigation after preventing the attack, *Wazuh* also successfully monitors the internal activities:

- **Privilege Escalation:** When the admin takes root access to use *sudo* iptables or any other command, *Wazuh* records it as "*Successful sudo to ROOT executed*". This is included in the *MITRE T1548.003* technique.



- **Audit Trail:** The log shows that a *PAM:* Login session opened/closed event is generated for each *sudo* command. This can be used to confirm whether an insider (internal) user is making any changes to the system.





Apr 13, 2026 @ 13:28:10.972	001	hasib			PAM: Login session closed.	3	5502
Apr 13, 2026 @ 13:28:10.953	001	hasib	T1078	Defense Evasion, Persistence, Privilege Escalation, Initial Access	PAM: Login session opened.	3	5501
Apr 13, 2026 @ 13:28:10.953	001	hasib	T1548.003	Privilege Escalation, Defense Evasion	Successful sudo to ROOT executed.	3	5402

Administrative Activity Summary:

Timestamp	Event Description	Alert Level	Security Impact
13:28:10	**Successful sudo to ROOT**	3	An important task was performed using admin privileges.
13:28:10	**PAM: Login session opened**	3	A new administrative session has started on the system.

Wazuh is a powerful *EDR/XDR* solution. It is capable of detecting and blocking external threats, while maintaining transparency of internal administrative work and preserving audit trails.

Malware Detection (ClamAV)

Installing ClamAV on Ubuntu VM:

Command:

sudo apt install -y clamav clamav-daemon

```
osboxes@hasib:~$ sudo apt install -y clamav clamav-daemon
[sudo] password for osboxes:
Sorry, try again.
[sudo] password for osboxes:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  clamav-base clamav-freshclam clamdscan libclamav12
Suggested packages:
  libclamunrar clamav-doc daemon libclamunrar11
The following NEW packages will be installed:
  clamav clamav-base clamav-daemon clamav-freshclam clamdscan libclamav12
0 upgraded, 6 newly installed, 0 to remove and 619 not upgraded.
Need to get 6,886 kB of archives.
After this operation, 32.4 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 clamav-base all 1.4.3+dfsg-0ubuntu0.24.04.1 [102 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libclamav12 amd64 1.4.3+dfsg-0ubuntu0.24.04.1 [3,598 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 clamav-freshclam amd64 1.4.3+dfsg-0ubuntu0.24.04.1 [99.1 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 clamav-daemon amd64 1.4.3+dfsg-0ubuntu0.24.04.1 [216 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 clamav amd64 1.4.3+dfsg-0ubuntu0.24.04.1 [2,819 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 clamdscan amd64 1.4.3+dfsg-0ubuntu0.24.04.1 [51.7 kB]
Fetched 6,886 kB in 22s (315 kB/s)
Preconfiguring packages ...
Selecting previously unselected package clamav-base.
(Reading database ... 152535 files and directories currently installed.)
Preparing to unpack .../0-clamav-base_1.4.3+dfsg-0ubuntu0.24.04.1_all.deb ...
Unpacking clamav-base (1.4.3+dfsg-0ubuntu0.24.04.1) ...
Selecting previously unselected package libclamav12:amd64.
Preparing to unpack .../1-libclamav12_1.4.3+dfsg-0ubuntu0.24.04.1_amd64.deb ...
Unpacking libclamav12:amd64 (1.4.3+dfsg-0ubuntu0.24.04.1) ...
Selecting previously unselected package clamav-freshclam
```

Turn off automatic updater:

```
osboxes@hasib:~$ sudo systemctl stop clamav-freshclam
```

Updating the virus database:

Command:

```
osboxes@hasib:~$ sudo freshclam
```

output:

```
osboxes@hasib:~$ sudo freshclam
ClamAV update process started at Mon Apr 13 21:25:21 2026
WARNING: Mon Apr 13 21:25:21 2026 -> FreshClam previously received error code 429 or 403 from the ClamAV Content Delivery Network (CDN).
Mon Apr 13 21:25:21 2026 -> This means that you have been rate limited or blocked by the CDN.
Mon Apr 13 21:25:21 2026 ->  1. Verify that you're running a supported ClamAV version.
Mon Apr 13 21:25:21 2026 ->     See https://docs.clamav.net/faq/faq-eol.html for details.
Mon Apr 13 21:25:21 2026 ->  2. Run FreshClam no more than once an hour to check for updates.
Mon Apr 13 21:25:21 2026 ->     FreshClam should check DNS first to see if an update is needed.
Mon Apr 13 21:25:21 2026 ->  3. If you have more than 10 hosts on your network attempting to download,
Mon Apr 13 21:25:21 2026 ->     it is recommended that you set up a private mirror on your network using
Mon Apr 13 21:25:21 2026 ->     cvdupdate (https://pypi.org/project/cvdupdate/) to save bandwidth on the
Mon Apr 13 21:25:21 2026 ->     CDN and your own network.
Mon Apr 13 21:25:21 2026 ->  4. Please do not open a ticket asking for an exemption from the rate limit,
Mon Apr 13 21:25:21 2026 ->     it will not be granted.
Mon Apr 13 21:25:21 2026 ->  5. If you have verified that you are not blocked due to your region, and have
Mon Apr 13 21:25:21 2026 ->     not exceeded the rate limit, please provide the following cf-ray id when
Mon Apr 13 21:25:21 2026 ->     submitting a ticket: 9ebc826c88102dfa-ARN
Mon Apr 13 21:25:21 2026 ->
WARNING: Mon Apr 13 21:25:21 2026 -> You are still on cool-down until after: 2026-04-14 21:24:39
```

Starting the ClamAV daemon:

```
osboxes@hasib:~$ sudo systemctl start clamav-daemon
```

Then enable:

Command:

sudo systemctl enable clamav-daemon

```
osboxes@hasib:~$ sudo systemctl enable clamav-daemon
Synchronizing state of clamav-daemon.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable clamav-daemon
```

Checking Status:

```
osboxes@hasib:~$ sudo systemctl status clamav-daemon
```

output:

```
osboxes@hasib:~$ sudo systemctl status clamav-daemon
○ clamav-daemon.service - Clam AntiVirus userspace daemon
     Loaded: loaded (/usr/lib/systemd/system/clamav-daemon.service; enabled; preset: enabled)
    Drop-In: /etc/systemd/system/clamav-daemon.service.d
             └─extend.conf
     Active: inactive (dead)
TriggeredBy: ○ clamav-daemon.socket
  Condition: start condition unmet at Mon 2026-04-13 21:25:40 EDT; 23s ago
       Docs: man:clamd(8)
             man:clamd.conf(5)
             https://docs.clamav.net/

Apr 13 21:24:41 hasib systemd[1]: clamav-daemon.service - Clam AntiVirus userspace daemon was skipped because of an unmet condition check (ConditionPathExistsGlob>
Apr 13 21:25:40 hasib systemd[1]: clamav-daemon.service - Clam AntiVirus userspace daemon was skipped because of an unmet condition check (ConditionPathExistsGlob>
```

Analyzing the output of *freshclam* and *systemctl* status above shows two problems:

- **Rate Limited (Error 429/403):** Here, it is possible that *freshclam* has been run repeatedly or the *ClamAV* server has blocked me (*Cool-down*) for 24 hours due to network problems.
- **Daemon Inactive (Condition unmet):** Since the virus database (*main.cvd* or *daily.cvd* file) has not been downloaded yet, the *ClamAV* service is unable to start. That is why the status shows inactive (*dead*).

Config for sending ClamAV Log to Wazuh:

```
osboxes@hasib:~$ sudo nano /var/ossec/etc/ossec.conf
```

Add inside <ossec_config>:

```
 GNU nano 7.2                                        /var/ossec/etc/ossec.conf *
    <enabled>yes</enabled>
    <interval>5m</interval>
    <max_eps>10</max_eps>
  </synchronization>
 </syscheck>

 <!-- Log analysis -->
 <localfile>
   <log_format>command</log_format>
   <command>df -P</command>
   <frequency>360</frequency>
 </localfile>

 <localfile>
   <log_format>full_command</log_format>
   <command>netstat -tulpn | sed 's/\([[:alnum:]]\+\)\ \+[[:digit:]]\+\ \+[[:digit:]]\+\ \+\(.*\):\([[:digit:]]*\)\ \+\([0-9\.\:\*]\+\).\+\ \([[:digit:]]*\/[[:al
   <alias>netstat listening ports</alias>
   <frequency>360</frequency>
 </localfile>

<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/clamav/clamav.log</location>
</localfile>

 <localfile>
   <log_format>full_command</log_format>
   <command>last -n 20</command>

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark   M-] To Bracket  M-Q Previous
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy       ^Q Where Was    M-W Next
```

Restarting the Agent:

```
osboxes@hasib:~$ sudo systemctl restart wazuh-agent
```

Simulating Malware with an EICAR Test File:

EICAR is a safe test string - all antiviruses detect it as a "*virus*". Not a real virus.

Creating a custom signature file:

```
osboxes@hasib:~$ echo "Eicar-Test-Signature:0:0:58354f2150254041505b345c505a58353428505e2937434329377d2445494341522d5354414e444152442d414e544956495255532d544553542
d46494c452124482b482a" | sudo tee /var/lib/clamav/manual.ndb
Eicar-Test-Signature:0:0:58354f2150254041505b345c505a58353428505e2937434329377d2445494341522d5354414e444152442d414e544956495255532d544553542d46494c452124482b482a
```

Setting permissions:

```
osboxes@hasib:~$ sudo chown clamav:clamav /var/lib/clamav/manual.ndb
```

First, making sure that the EEC test file exists:

```
osboxes@hasib:~$ echo 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' > /tmp/eicar_test.txt
```

Single file scan:

```
osboxes@hasib:~$ sudo clamscan /tmp/eicar_test.txt
```

output:

```
osboxes@hasib:~$ sudo clamscan /tmp/eicar_test.txt
[sudo] password for osboxes:
Loading:     0s, ETA:   0s [=========================>]        1/1 sigs
Compiling:   0s, ETA:   0s [=========================>]       40/40 tasks

/tmp/eicar_test.txt: Eicar-Test-Signature.UNOFFICIAL FOUND

----------- SCAN SUMMARY -----------
Known viruses: 1
Engine version: 1.4.3
Scanned directories: 0
Scanned files: 1
Infected files: 1
Data scanned: 0.00 MB
Data read: 0.00 MB (ratio 0.00:1)
Time: 0.054 sec (0 m 0 s)
Start Date: 2026:04:13 23:32:06
End Date:   2026:04:13 23:32:06
```

Scan the entire /tmp folder and write to the log:

```
osboxes@hasib:~$ sudo clamscan -d /var/lib/clamav/manual.ndb -r /tmp/ --log=/var/log/clamav/clamav.log --infected
```

Output:

```
osboxes@hasib:~$ sudo clamscan -d /var/lib/clamav/manual.ndb -r /tmp/ --log=/var/log/clamav/clamav.log --infected
/tmp/eicar_test.txt: Eicar-Test-Signature.UNOFFICIAL FOUND

----------- SCAN SUMMARY -----------
Known viruses: 1
Engine version: 1.4.3
Scanned directories: 55
Scanned files: 6
Infected files: 1
Data scanned: 0.00 MB
Data read: 0.00 MB (ratio 0.00:1)
Time: 0.016 sec (0 m 0 s)
Start Date: 2026:04:13 21:51:42
End Date:   2026:04:13 21:51:42
```

The output shows that,

Scanned files: 6

Infected files: 1

View log:

Command:

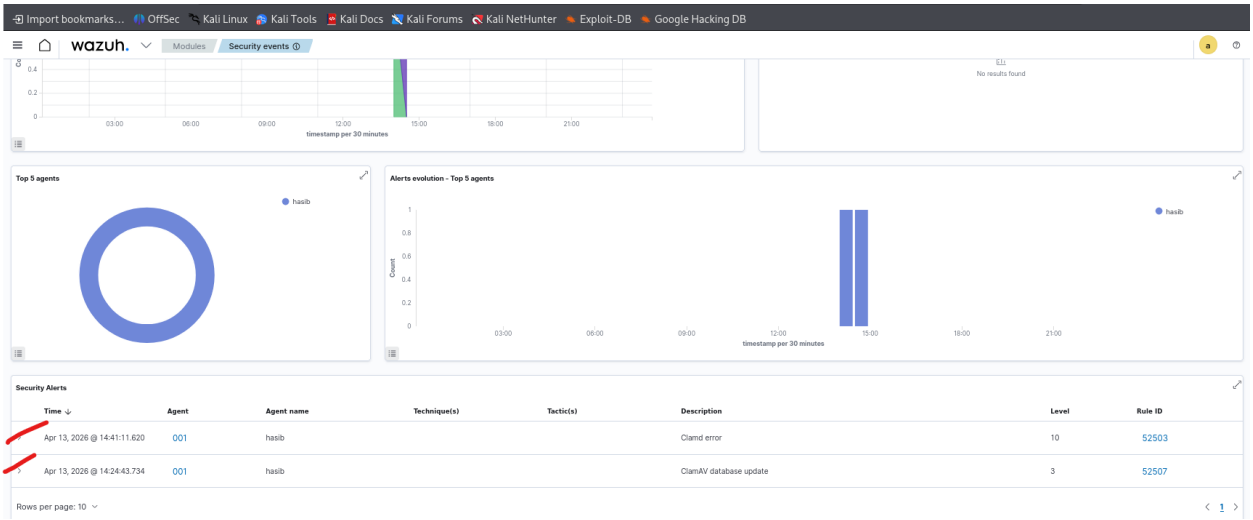
Cat /var/log/clamav/clamav.log | grep FOUND

Output:

```
osboxes@hasib:~$ cat /var/log/clamav/clamav.log | grep FOUND
Mon Apr 13 21:50:00 2026 -> /tmp/eicar.com: Eicar-Test-Signature FOUND
/tmp/eicar_test.txt: Eicar-Test-Signature.UNOFFICIAL FOUND
```

Infected files: 1

I search for malware or virus in Security Events from the Dashboard, an alert appears.



1. Scan Execution Tracking

According to the logs available in the *Wazuh* dashboard, the system successfully recorded the *ClamAV* scan run by the administrator.

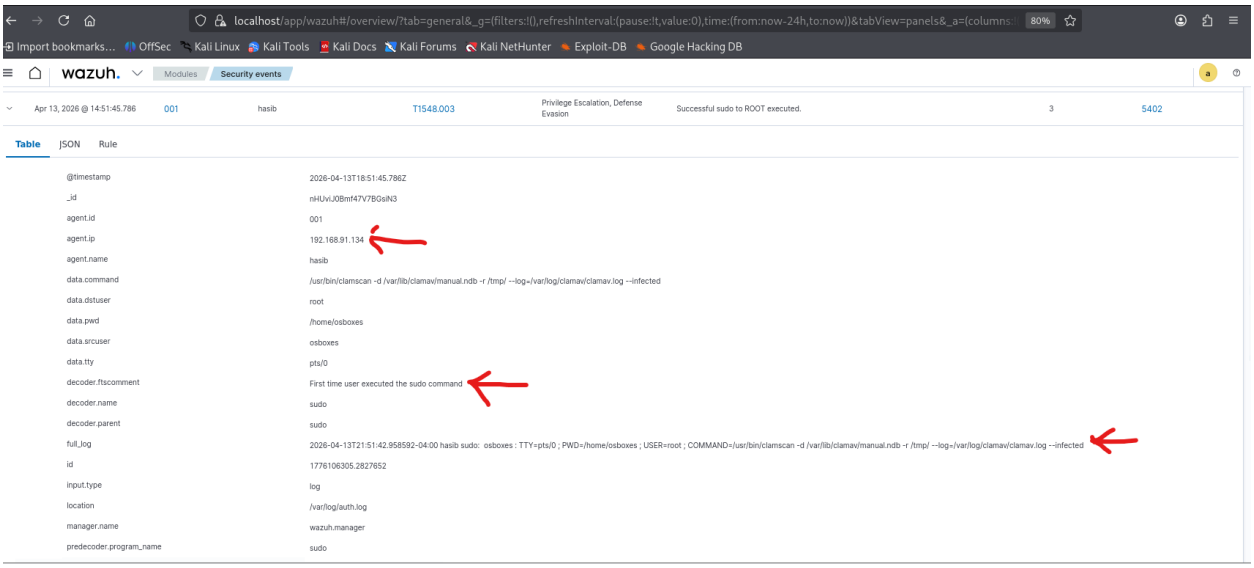
- **Event:** Successful *sudo* to *ROOT* executed.
- **Timestamp:** Apr 13, 2026 @ 14:51:45
- **Action:** *Wazuh* detected in real-time that the */usr/bin/clamscan* command was scanning the */tmp/* directory using a custom database (*manual.ndb*).

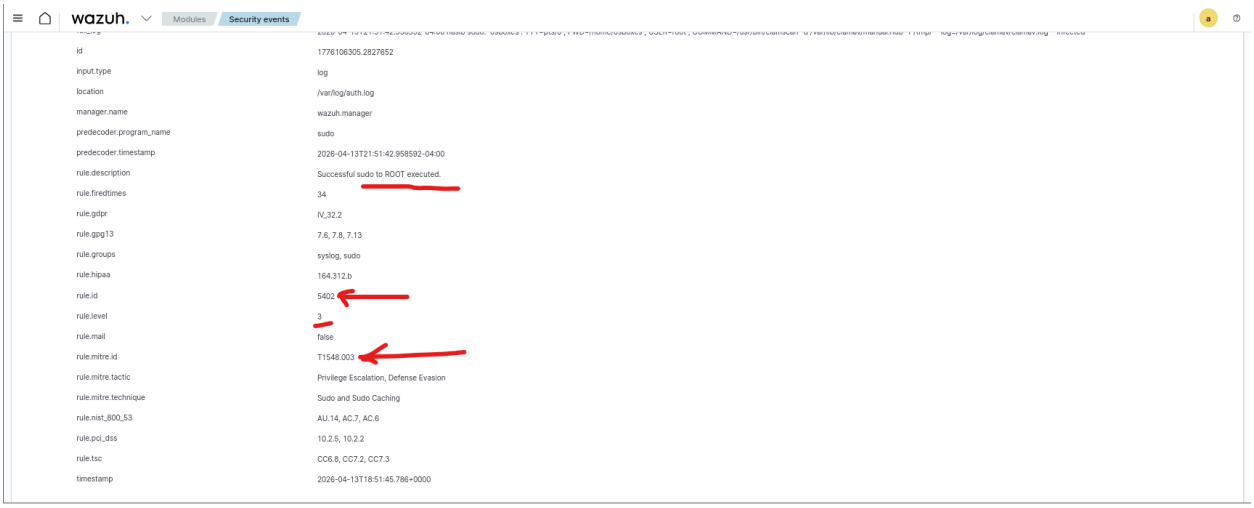


2. MITRE ATT&CK Mapping

Wazuh has classified this event as a *T1548.003* (*Sudo* and *Sudo Caching*) technique. It demonstrates that:

- **Privilege Escalation**: A security scan was run with root privileges from user *osboxes.*
- **Audit Trail:** A full log of each scanning command is stored in the *SIEM* system, which aids in forensic analysis.





File Integrity Monitoring (FIM) test

The configuration file was opened from the *Linux Agent* in the terminal:

```
osboxes@hasib:~$ sudo nano /var/ossec/etc/ossec.conf
```

Find the <*syscheck*> section in the file and confirm the following line there:

```
<syscheck>
  <disabled>no</disabled>

  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

  <scan_on_start>yes</scan_on_start>

  <!-- Directories to check  (perform all possible verifications) -->
  <directories>/etc,/usr/bin,/usr/sbin</directories>
  <directories>/bin,/sbin,/boot</directories>
  <directories check_all="yes" report_changes="yes" realtime="yes">/etc</directories>

  <!-- Files/directories to ignore -->
  <ignore>/etc/mtab</ignore>
  <ignore>/etc/hosts.deny</ignore>
  <ignore>/etc/mail/statistics</ignore>
  <ignore>/etc/random-seed</ignore>
  <ignore>/etc/random.seed</ignore>
  <ignore>/etc/adjtime</ignore>
  <ignore>/etc/httpd/logs</ignore>
  <ignore>/etc/utmpx</ignore>
  <ignore>/etc/wtmpx</ignore>
  <ignore>/etc/cups/certs</ignore>
  <ignore>/etc/dumpdates</ignore>
  <ignore>/etc/svc/volatile</ignore>

  <!-- File types to ignore -->
```

Here, *realtime="yes*" means that alerts are received as soon as the file changes. The file is saved and the agent is restarted:

```
osboxes@hasib:~$ sudo systemctl restart wazuh-agent
```

Running a File Attack Simulation

Now run my commands one by one in the terminal. I wait 1-2 minutes after running each command so that *Wazuh* can process the data.

Create a new file (Added)

Command:

echo "test config" | sudo tee /etc/test_fim.conf

output:

```
osboxes@hasib:~$ echo "test config" | sudo tee /etc/test_fim.conf
test config
```

File modified

Command:

echo "MODIFIED - attacker was here" | sudo tee /etc/test_fim.conf

output:

```
osboxes@hasib:~$ echo "MODIFIED - attacker was here" | sudo tee /etc/test_fim.conf
MODIFIED - attacker was here
```

Permission Change

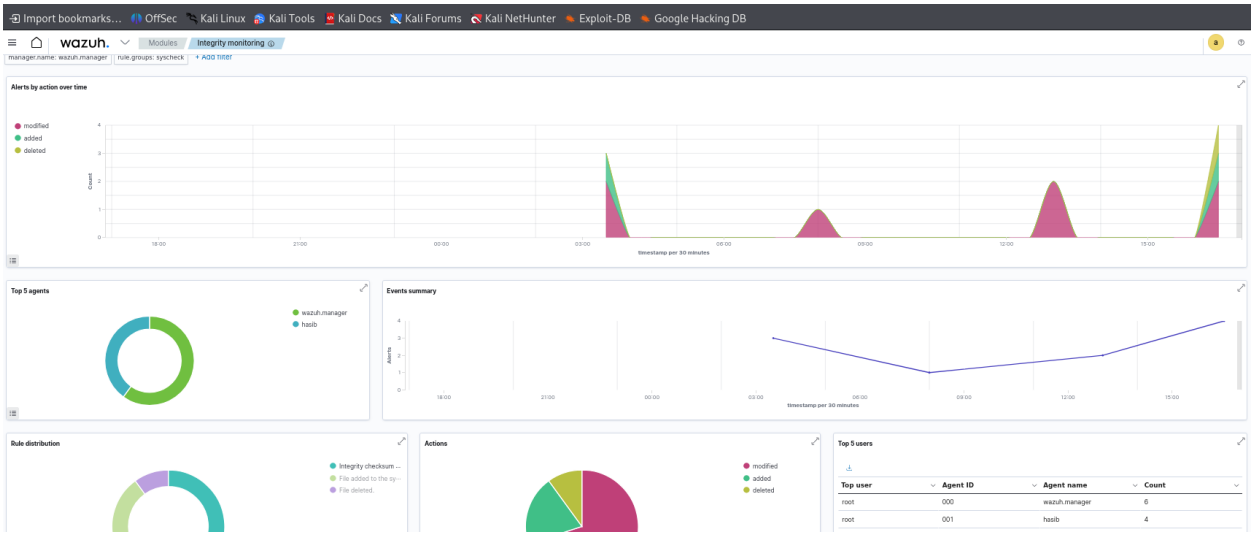
```
osboxes@hasib:~$ sudo chmod 777 /etc/test_fim.conf
```

File deleted (Deleted)

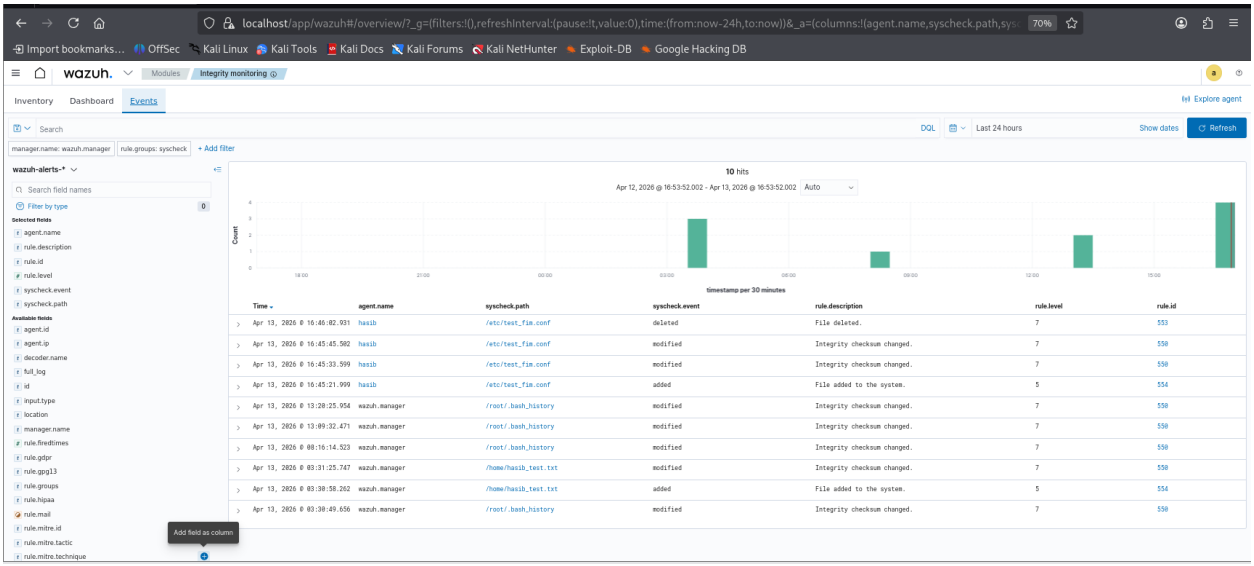
```
osboxes@hasib:~$ sudo rm /etc/test_fim.conf
```

Verification in Wazuh Dashboard

I go to *Wazuh Dashboard* and click on *Integrity Monitoring* from *Modules* in the left menu.



Clicking on the *Events* tab will show you the alerts labeled *Added, Modified,* and *Deleted.*

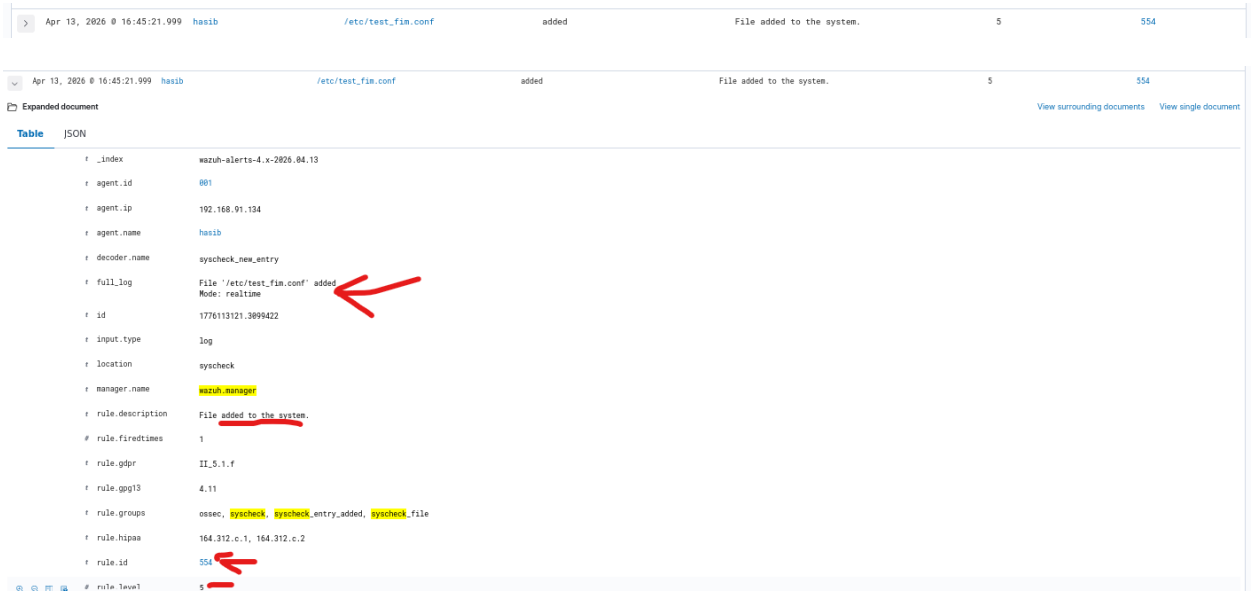


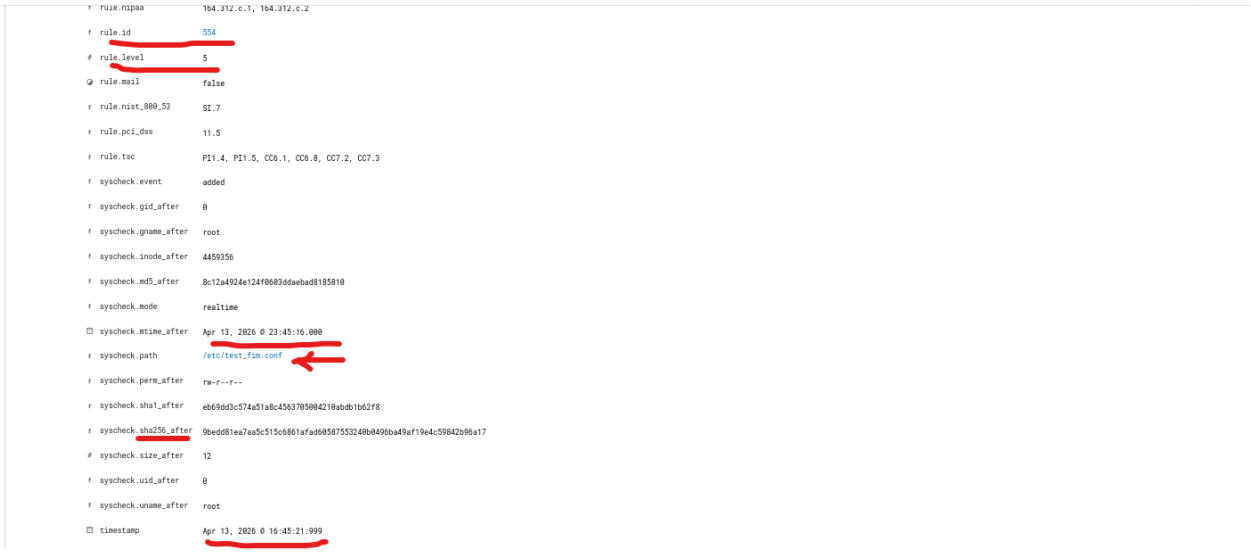
File Integrity Monitoring (FIM) Result Analysis

Using the *Syscheck* module of *Wazuh*, I monitored the lifecycle of the */etc/test_fim.conf* file in real-time. The results obtained from the log are given below:

1. File Added Detection

- **Timestamp:** Apr 13, 2026 @ 16:45:21
- **Rule ID:** 554 (Level 5)
- **Observation:** As soon as a new file is created in the */etc/ directory, Wazuh* detects it and saves the initial hash value (*MD5/SHA256*) of the file in the database.

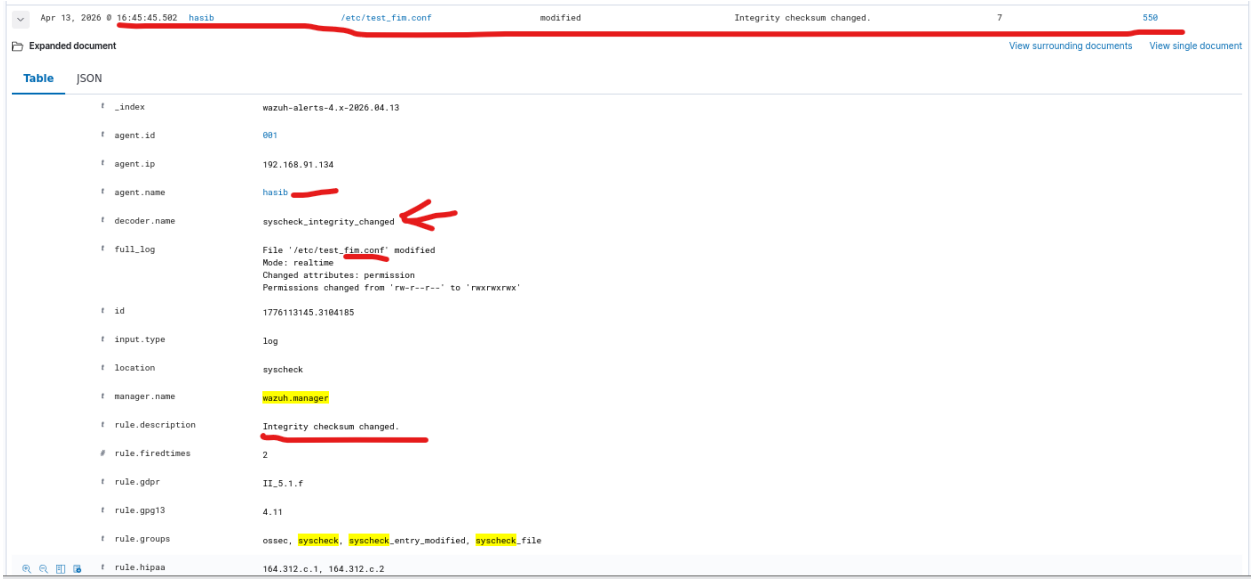


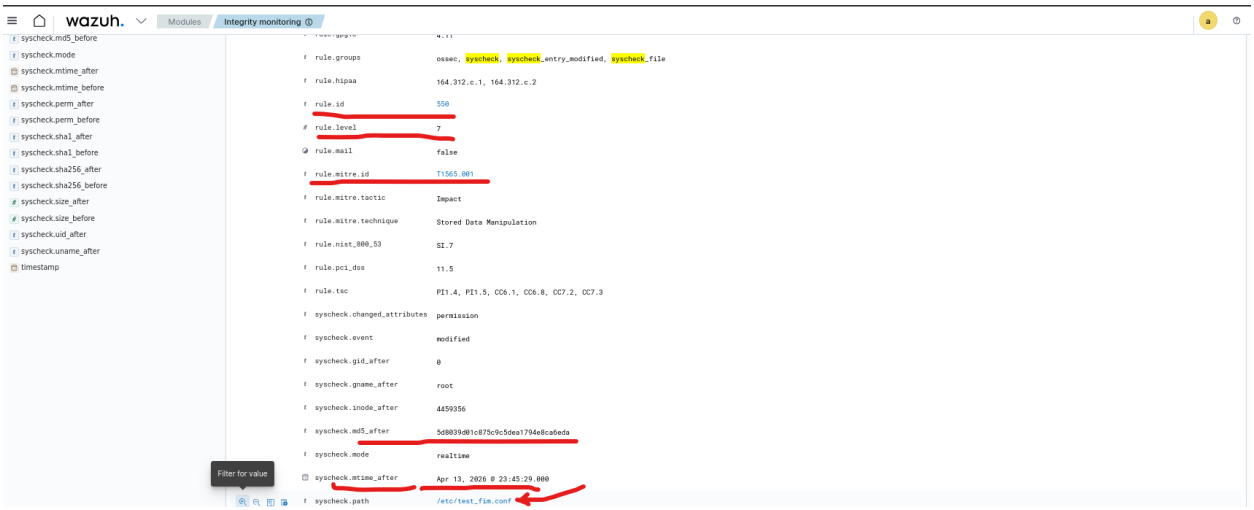


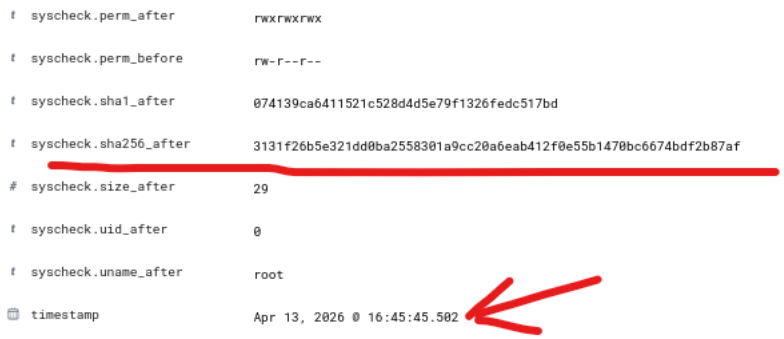
2. Content Modification and Diff Analysis (Content Modified)

This is the most important part of this lab. Wazuh not only reported that the file was modified, but also showed what was changed:

- **Rule ID:** 550 (Level 7)
- **Diff Analysis:** Old Content: test config
  - **New Content:** *MODIFIED* - attacker was here
- **Metadata Changes:** The file size increased from 12 to 29 bytes and the checksum changed.





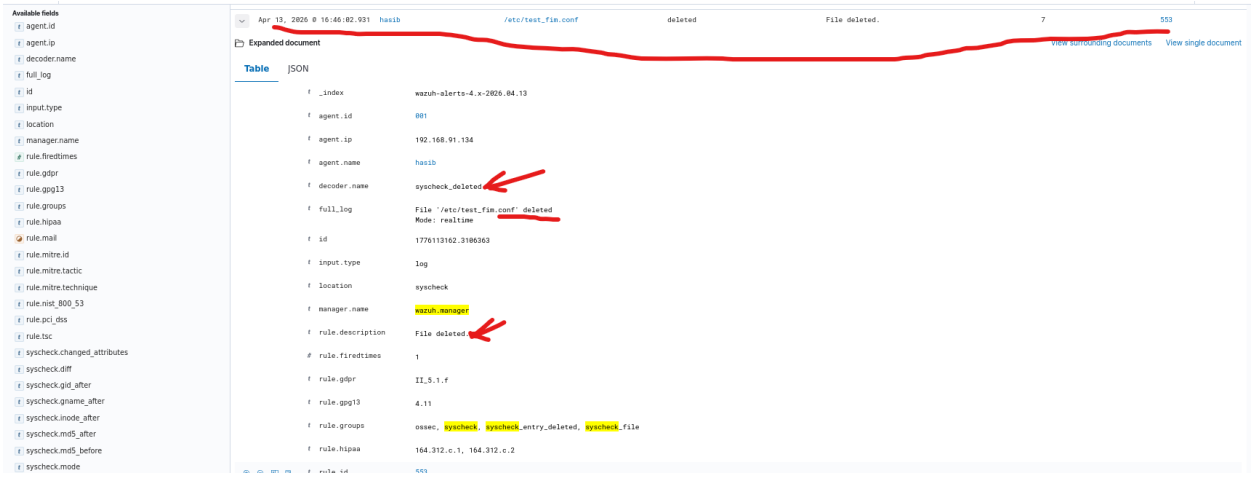


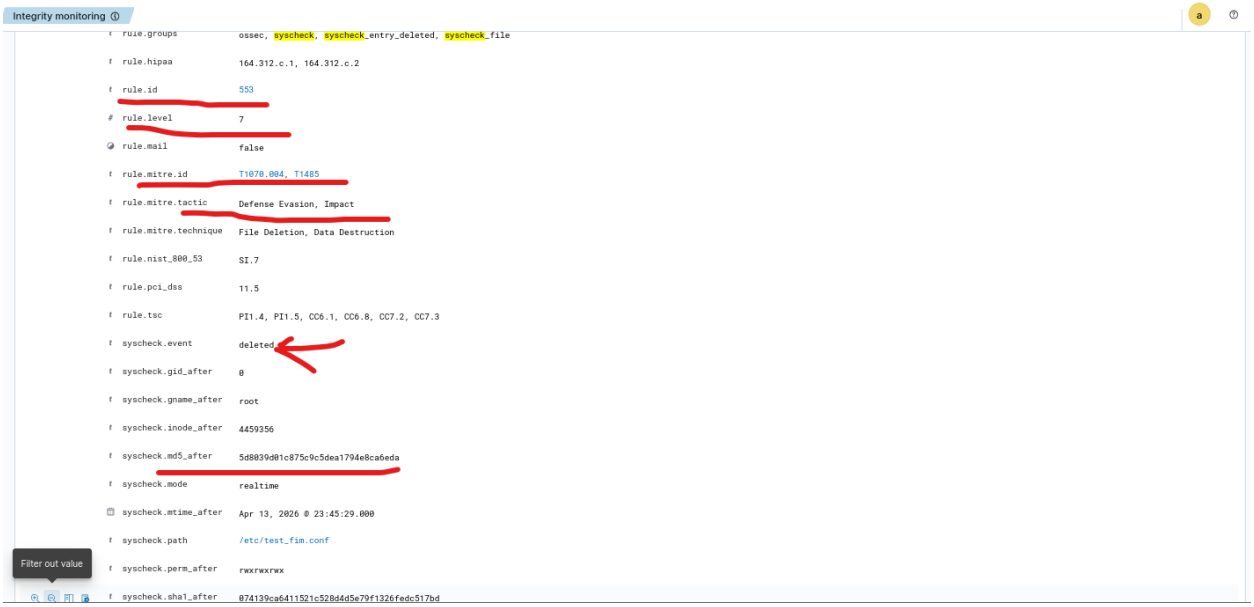
3. Permission Integrity

- **Observation:** The file's permissions were changed from *rw-r--r-- to rwxrwxrwx (777). Wazuh* specifically detected this change, which indicates an unauthorized privilege change.

4. File Delete Detection

- **Timestamp:** Apr 13, 2026 @ 16:46:02
- **Rule ID:** 553 (Level 7)
- **Observation:** *Wazuh* generated an alert in real-time as soon as the file was deleted.







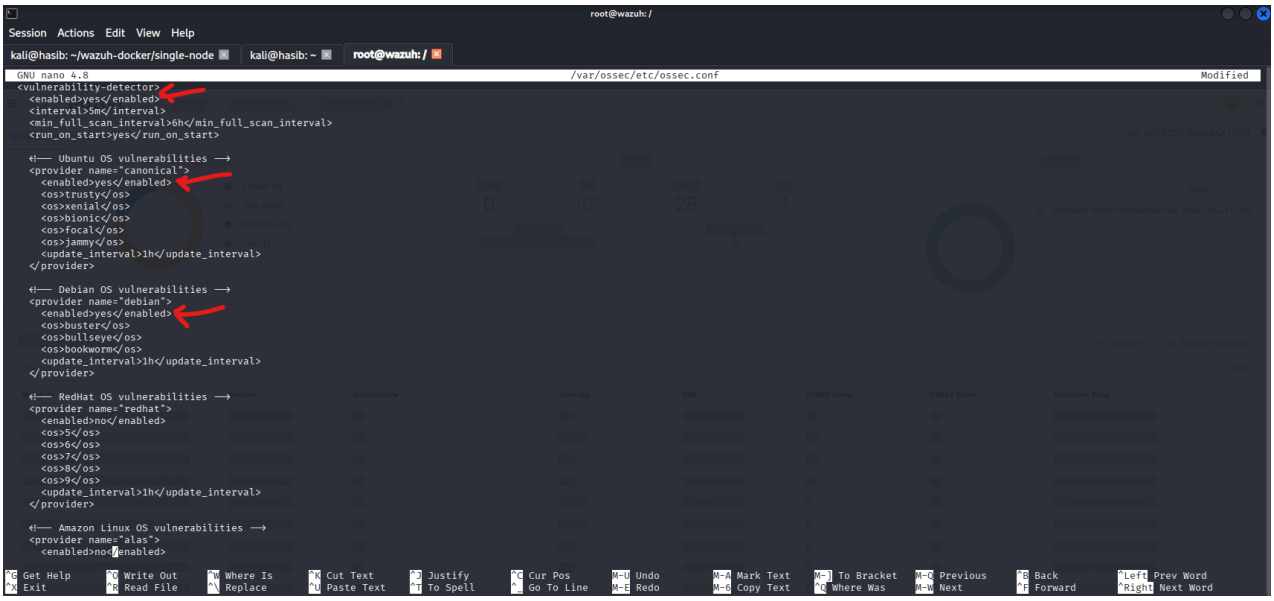
FIM

Time	Event	File Path	Change Detected	Rule ID
16:45:21	**Added**	/etc/test_fim.conf	New entry created	554

16:45:33	**Modified**	/etc/test_fim.conf	Content changed (Diff recorded)	550
16:45:45	**Modified**	/etc/test_fim.conf	Permissions changed to 777	550
16:46:02	**Deleted**	/etc/test_fim.conf	File removed from system	553

Vulnerability Detection

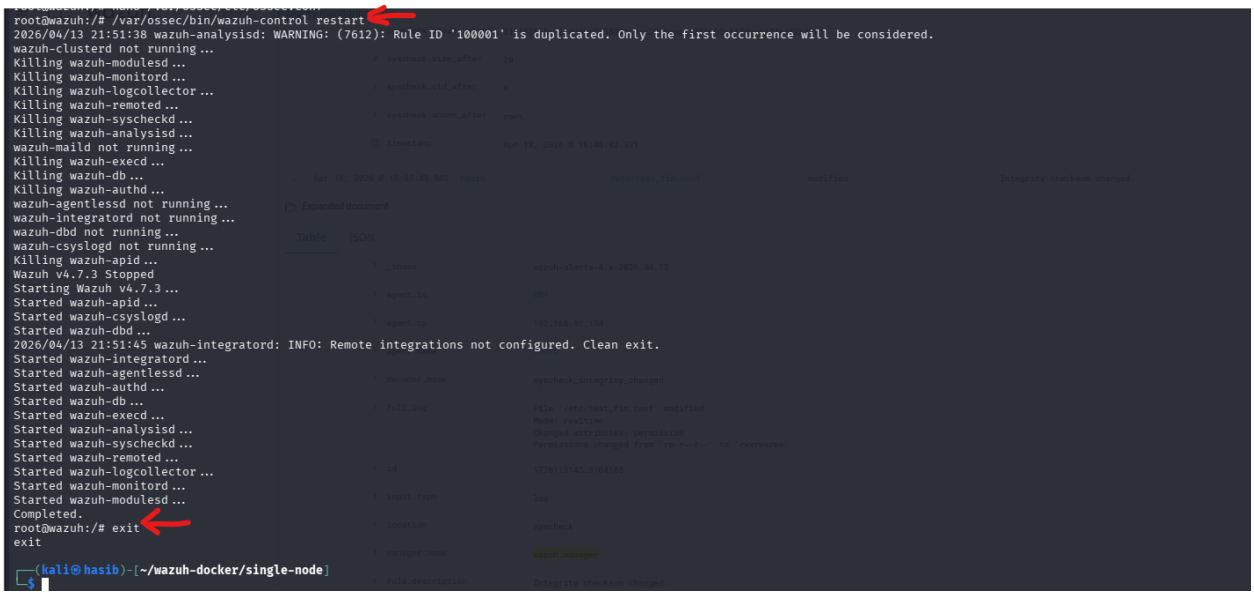
I go inside *single-node-wazuh.manager-1* and open the *ossec.conf* file and change *<vulnerability-detector>* to *<enabled>yes</enabled>,* here enabled "*yes*". Then I go to *Debian OS* vulnerabilities and enable "*yes*".



Then, wazuh-control was restarted with the following command:

Command:

/var/ossec/bin/wazuh-control restart



Then I exit.

The Vulnerabilities menu from the Dashboard shows a list of detected CVEs according to severity.

Vulnerability Detection Analysis

The vulnerabilities in the system were identified by scanning the endpoint inventory through *Wazuh SIEM*. In this test, a total of 130 security vulnerabilities were found in the *Agent 002* (*Windows 11*) endpoint, out of which 101 were high, 28 were medium, and 1 was low.

1. Key Findings

The scan report shows that there are several high-level risks in the *Microsoft Office Professional Plus 2016* application installed on the system:

- **High Severity CVEs**: Some of the notable *CVEs* are CVE-2024-30104 and *CVE-2024-30101*. These mainly pose a remote code execution (*RCE*) or information disclosure risk.
- **CVSS Scores:** The detected threats have *CVSS 3.0* scores ranging from *7.5* to *7.8,* indicating that they are highly dangerous and require immediate resolution.

2. Detection Capability (Detection Mechanism)

- **Real-time Monitoring:** The scan was successfully completed on *April 13, 2026.*
- **Comprehensive Coverage:** The *SIEM* system is capable of detecting vulnerabilities not only in the *OS*, but also in third-party applications (e.g. *MS Office*).

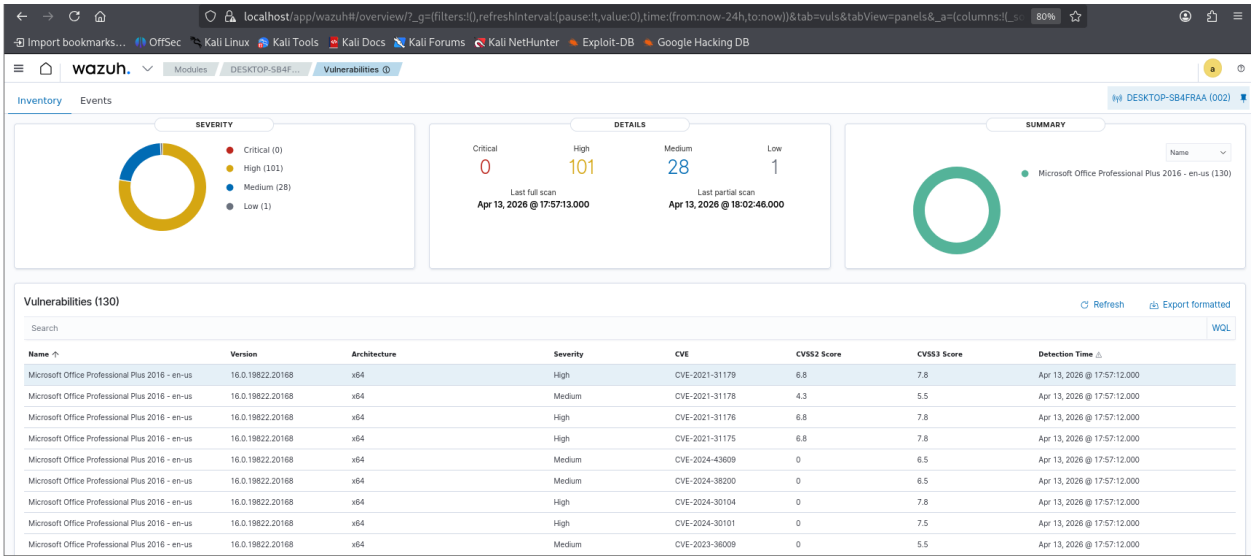
3. Recommendations

The following steps should be taken based on the report:

1. Update the affected applications with the latest security patches.

2. Remove software that is no longer supported (*End-of-life*) from the system.

3. Schedule regular automated scans of critical endpoints.



Incident Response and Security Reports

Getting to the right dashboard

First, I went from *Wazuh* to *Security Events* from the left menu.

Selecting the time range

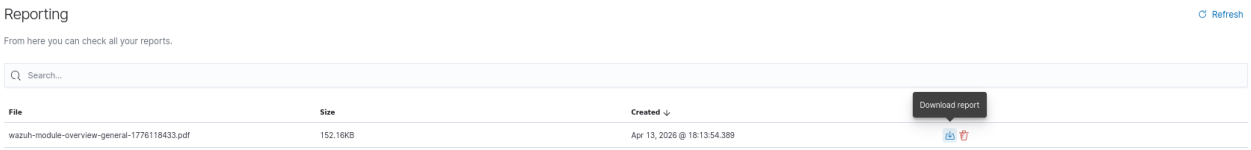
Selecting "*Last 24 hours*" from the calendar icon in the top right corner. This brings up all the data from my brute force, malware scan, and *FIM* to the report.

Generating a report with one click

As soon as I clicked on Generate report at the top right of the dashboard, a notification appeared on the screen: "*Reporting: Report is being generated*".

Downloading

Once the report was generated, I went from *Management* to *Reporting* from the left menu. There I could see the PDF file I had created, which was then downloaded.



Security Events Report Analysis

A complete security report for the last 24 hours has been generated using the *Wazuh SIEM* module. The salient features of the report are given below:

1. Alerts Summary

According to the report, alerts of different categories have been identified in the system:

- **High Severity Alerts (Level 10):** Several critical vulnerabilities have been identified in the system, including *CVE-2016-0025* and *CVE-2017-11882*. These are mainly related to *Microsoft Office* applications.
- **Medium Severity Alerts (Level 7):** About 19 different categories of level *7* alerts have been found. Notable among these are security flaws related to Interactive *Logon Message Configuration* and *Account Lockout Threshold.*
- **Compliance & Benchmarks:** The report displays configuration audit results for *Ubuntu* and *Windows 11* as per *CIS (Center for Internet Security)* benchmarks (e.g. *SSH MaxAuthTries, AppArmor Enablement*).

2. Critical Findings

- **Windows Logon Success:** The report records 186 successful login events, which helps in tracking user activity.
- **Vulnerability Mapping:** The SIEM system directly reports the vulnerabilities of installed software using *CVE IDs* (e.g. *CVE-2021-3995*).

wazuh.

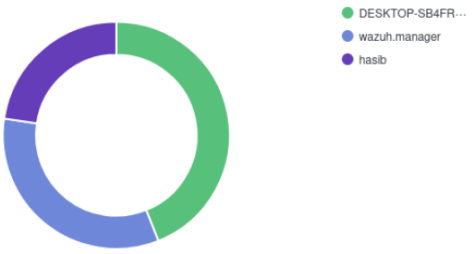
info@wazuh.com
https://wazuh.com

Security events report

Browse through your security alerts, identifying issues and threats in your environment.

2026-04-12T18:13:52 to 2026-04-13T18:13:52
manager.name: wazuh.manager

Top 5 agents



Alerts



Copyright © 2023 Wazuh, Inc.

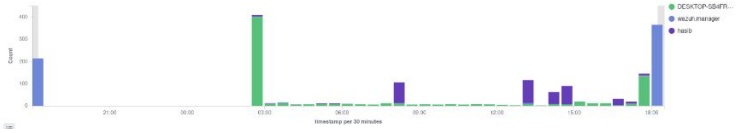
wazuh.

info@wazuh.com
https://wazuh.com

Alert level evolution



Alerts evolution Top 5 agents



Alerts summary

Rule ID	Description	Level	Count
60106	Windows logon success.	3	186
19008	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Allow Basic authentication' is set to 'Disabled'.	3	2
19008	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Allow unencrypted traffic' is set to 'Disabled'.	3	2
19007	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Configure 'Accounts: Rename administrator account'.	7	1
19007	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Configure 'Accounts: Rename guest account'.	7	1
19007	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Configure 'Interactive logon: Message text for users attempting to log on'.	7	1
19007	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Configure 'Interactive logon: Message title for users attempting to log on'.	7	1
19007	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Disable IPv6 (Ensure TCPIP6 Parameter 'DisabledComponents' is set to '0xff (255)').	7	1
19007	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Account lockout duration' is set to '15 or more minute(s)'.	7	1
19007	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Account lockout threshold' is set to '5 or fewer invalid logon attempt(s), but not 0'.	7	1
19007	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Accounts: Block Microsoft accounts' is set to 'Users can't add or log on with Microsoft accounts'.	7	1
19007	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Allow Clipboard synchronization across devices' is set to 'Disabled'.	7	1
19007	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Allow Cloud Search' is set to 'Enabled: Disable Cloud Search'.	7	1
19007	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Allow Cortana above lock screen' is set to 'Disabled'.	7	1
19007	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Allow Cortana' is set to 'Disabled'.	7	1
19007	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Allow Diagnostic Data' is set to 'Enabled: Diagnostic	7	1

wazuh.

info@wazuh.com
https://wazuh.com

Rule ID	Description	Level	Count
23504	CVE-2019-1084 affects Microsoft Office Professional Plus 2016 - en-us	7	1
23504	CVE-2019-1400 affects Microsoft Office Professional Plus 2016 - en-us	7	1
23504	CVE-2019-1402 affects Microsoft Office Professional Plus 2016 - en-us	7	1
23504	CVE-2019-1446 affects Microsoft Office Professional Plus 2016 - en-us	7	1
23504	CVE-2019-1463 affects Microsoft Office Professional Plus 2016 - en-us	7	1
23504	CVE-2019-1464 affects Microsoft Office Professional Plus 2016 - en-us	7	1
23504	CVE-2020-1229 affects Microsoft Office Professional Plus 2016 - en-us	7	1
23504	CVE-2020-1497 affects Microsoft Office Professional Plus 2016 - en-us	7	1
23504	CVE-2021-31174 affects Microsoft Office Professional Plus 2016 - en-us	7	1
23504	CVE-2021-31178 affects Microsoft Office Professional Plus 2016 - en-us	7	1
23504	CVE-2021-3995 affects bsdutils	7	1
23504	CVE-2021-3996 affects bsdutils	7	1
23504	CVE-2021-40454 affects Microsoft Office Professional Plus 2016 - en-us	7	1
19008	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Accounts: Administrator account status' is set to 'Disabled'.	3	1
19008	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Accounts: Guest account status' is set to 'Disabled'.	3	1
19008	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Accounts: Limit local account use of blank passwords to console logon only' is set to 'Enabled'.	3	1
19008	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Allow a Windows app to share application data between users' is set to 'Disabled'.	3	1
19008	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Allow indexing of encrypted files' is set to 'Disabled'.	3	1
19008	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Allow remote server management through WinRM' is set to 'Disabled'.	3	1
19008	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Allow user control over installs' is set to 'Disabled'.	3	1
19008	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Allow users to connect remotely by using Remote Desktop Services' is set to 'Disabled'	3	1

	'Disabled'.		
19009	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Peer Networking Identity Manager (p2pimsvc)' is set to 'Disabled'.	3	1
19009	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Select when Quality Updates are received' is set to 'Enabled: 0 days'.	3	1
19009	CIS Microsoft Windows 11 Enterprise Benchmark v1.0.0: Ensure 'Support device authentication using certificate' is set to 'Enabled: Automatic'.	3	1
19009	CIS Ubuntu Linux 20.04 LTS Benchmark v2.0.0: Ensure AppArmor is enabled in the bootloader configuration.	3	1
19009	CIS Ubuntu Linux 20.04 LTS Benchmark v2.0.0: Ensure Automatic Error Reporting is not enabled.	3	1
19009	CIS Ubuntu Linux 20.04 LTS Benchmark v2.0.0: Ensure SSH AllowTcpForwarding is disabled.	3	1
19009	CIS Ubuntu Linux 20.04 LTS Benchmark v2.0.0: Ensure SSH HostbasedAuthentication is disabled.	3	1
19009	CIS Ubuntu Linux 20.04 LTS Benchmark v2.0.0: Ensure SSH Idle Timeout Interval is configured.	3	1
19009	CIS Ubuntu Linux 20.04 LTS Benchmark v2.0.0: Ensure SSH IgnoreRhosts is enabled.	3	1
19009	CIS Ubuntu Linux 20.04 LTS Benchmark v2.0.0: Ensure SSH LogLevel is appropriate.	3	1
19009	CIS Ubuntu Linux 20.04 LTS Benchmark v2.0.0: Ensure SSH LoginGraceTime is set to one minute or less.	3	1
19009	CIS Ubuntu Linux 20.04 LTS Benchmark v2.0.0: Ensure SSH MaxAuthTries is set to 4 or less.	3	1

Copyright © 2023 Wazuh, Inc.

wazuh.

info@wazuh.com
https://wazuh.com

Rule ID	Description	Level	Count
19009	CIS Ubuntu Linux 20.04 LTS Benchmark v2.0.0: Ensure SSH MaxSessions is set to 10 or less.	3	1

Task 4 - Log Management

Setup a log rotation for your SIEM system. Show on practice that it works. Explain how logs are rotated on Linux and how disk space is managed in relation to logs.

Bonus: show an advanced skills of Kibana & Wazuh log entities filtering. For example, prepare a scenario when you need to catch some security event in Wazuh using multiple or nested searching filters and expressions.

Bonus: correlate Wazuh security events with MITRE ATT&CK techniques.

Bonus: setup security alerts notifications for your SIEM system. Show on practice that it works.

Bonus: wrap your SIEM deployment as IaC within DevOps practices.

Solution:

How Linux Log Rotation Works

Log rotation

Log rotation is the process of automatically archiving, compressing or deleting old log files so that disk space does not fill up and the system does not slow down.

How it works: This is done by the *logrotate* tool in Linux. It reads the configuration from the */etc/logrotate.conf* and */etc/logrotate.d/* folders. Usually, the *cron* job runs it every day at 12 midnight.

Rotation Process (Example):

- **Today's log:** *syslog* (active, new entries are being added here)
- **If rotation occurs:**
  - *syslog* - renames to *syslog.1*
  - *syslog.1* - compresses to *syslog.2.gz*
  - *syslog.2.gz* - shifts to *syslog.3.gz*
  - creates a new empty *syslog*
  - *syslog.8.gz* is deleted if rotate is 7

In this lab, we tested system logs and disk space management on both *Kali Linux (SIEM Manager)* and *Ubuntu (Agent)* machines. Below are the details of each step and the commands used:

Wazuh Manager Log Analysis

The log mechanism of *Wazuh Manager* inside a *Docker* container running on a *Kali Linux* host was checked.

Container Identification:

First, the following command is used to find out the correct container name:

Command:

sudo docker ps --format "table {{.Names}}\t{{.Status}}"

Output:

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ sudo docker ps --format "table {{.Names}}\t{{.Status}}"
[sudo] password for kali:
NAMES                          STATUS
single-node-wazuh.dashboard-1   Up 16 hours
single-node-wazuh.indexer-1     Up 16 hours
single-node-wazuh.manager-1     Up 16 hours
```

The *single-node-wazuh.manager-1* container was successfully detected.

Manager log rotation check:

The following command is run to check the status of the logs inside the container:

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ sudo docker exec single-node-wazuh.manager-1 ls -lah /var/ossec/logs/
```

Output:

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ sudo docker exec single-node-wazuh.manager-1 ls -lah /var/ossec/logs/
total 348K
drwxrwx─── 8 wazuh wazuh 4.0K Apr 13 00:00 .
drwxr-x─── 1 root  wazuh 4.0K Apr 13 06:44 ..
-rw-rw──── 1 wazuh wazuh    0 Mar  4  2024 active-responses.log
drwxr-x─── 3 wazuh wazuh 4.0K Apr 13 22:03 alerts
drwxr-x─── 3 wazuh wazuh 4.0K Apr 13 00:00 api
-rw-rw──── 1 wazuh wazuh 214K Apr 13 23:00 api.log
drwxr-x─── 3 wazuh wazuh 4.0K Apr 13 22:03 archives
drwxr-x─── 2 wazuh wazuh 4.0K Feb 29  2024 cluster
-rw-rw──── 1 wazuh wazuh    0 Apr 12 22:13 cluster.log
drwxr-x─── 3 wazuh wazuh 4.0K Apr 13 22:03 firewall
-rw-r──── 1 wazuh wazuh    0 Mar  4  2024 integrations.log
-rw-rw──── 1 wazuh wazuh  85K Apr 13 22:58 ossec.log
drwxr-x─── 3 wazuh wazuh 4.0K Apr 13 00:00 wazuh
```

I see that there are folders like alerts, archives, etc. and the size of the ossec.log file is only *85 kilobytes*, which indicates that *Wazuh* is regularly archiving and rotating its logs.

Agent Disk Space and Log Analysis

Analyzed the status of the Ubuntu endpoint's disk storage and the default Linux log files.

Overall Disk Space Check:

The command is given to see if the system disk is getting full:

```
osboxes@hasib:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           392M  1.8M  390M   1% /run
/dev/sda2       492G   12G  455G   3% /
tmpfs           2.0G     0  2.0G   0% /dev/shm
tmpfs           5.0M     0  5.0M   0% /run/lock
tmpfs           392M  120K  392M   1% /run/user/1000
```

Out of the total *492 GB* of space on the */dev/sda2* partition, only *12 GB (3%)* is used, meaning the system is in a healthy state.

Determining the size of the log folder:

To see how much space the */var/log/ directory* is taking up:

```
osboxes@hasib:~$ sudo du -sh /var/log/
[sudo] password for osboxes:
226M    /var/log/
```

A total of *226 MB* of space is being used by log files.

Identifying the largest log files:

To find the top 15 largest log files for storage optimization:

```
osboxes@hasib:~$ sudo du -sh /var/log/* 2>/dev/null | sort -rh | head -15
```

Output:

```
osboxes@hasib:~$ sudo du -sh /var/log/* 2>/dev/null | sort -rh | head -15
217M    /var/log/journal
2.3M    /var/log/syslog
1.4M    /var/log/syslog.1
1.1M    /var/log/installer
844K    /var/log/kern.log
808K    /var/log/dpkg.log.1
660K    /var/log/sysstat
456K    /var/log/kern.log.1
144K    /var/log/auth.log
120K    /var/log/dmesg.0
116K    /var/log/dmesg
116K    /var/log/bootstrap.log
108K    /var/log/apt
84K     /var/log/cloud-init.log
80K     /var/log/boot.log.3
```

It can be seen that the journal (*217M*), syslog *(2.3M)* and *syslog.1 (1.4M)* files are taking up the most space.

Logrotate Configuration Verification

Evidence has been collected on how the *Linux* system is automatically managing logs.

Evidence of rotated and compressed logs:

To see how old logs are compressed *(Zip*) to save storage:

```
osboxes@hasib:~$ ls -lah /var/log/*.gz 2>/dev/null
```

Output:

```
osboxes@hasib:~$ ls -lah /var/log/*.gz 2>/dev/null
-rw-r----- 1 root adm 24K Apr 13 08:50 /var/log/dmesg.1.gz
-rw-r----- 1 root adm 24K Apr 13 08:49 /var/log/dmesg.2.gz
-rw-r----- 1 root adm 24K Apr 13 01:43 /var/log/dmesg.3.gz
-rw-r----- 1 root adm 24K Apr 12 16:30 /var/log/dmesg.4.gz
```

The presence of the *dmesg.1.gz, dmesg.2.gz* files proves that compression is working.

Policy or Rules Check:

The */etc/logrotate.d/rsyslog* file is checked to see which rules this rotation is following.

```
osboxes@hasib:~$ cat /etc/logrotate.d/rsyslog
```

Output:

```
osboxes@hasib:~$ cat /etc/logrotate.d/rsyslog
/var/log/syslog
/var/log/mail.log
/var/log/kern.log
/var/log/auth.log
/var/log/user.log
/var/log/cron.log
{
        rotate 4
        weekly
        missingok
        notifempty
        compress
        delaycompress
        sharedscripts
        postrotate
                /usr/lib/rsyslog/rsyslog-rotate
        endscript
}
```

The configuration has the weekly, *rotate 4*, and compress options enabled. This means that the logs will be rotated every week, keeping a maximum of 4 old files and compressing them to save storage.

Wazuh Log Rotation Configuration

Wazuh Log Rotation and Custom Policy Configuration

The task here is to analyze *Wazuh Manager's (SIEM)* own log rotation mechanism and ensure its functionality by creating a custom log rotation policy on the *Ubuntu* endpoint.

Wazuh Internal Rotation Analysis

This step examines how *Wazuh Manager* manages logs inside a Docker container.

Entering the Wazuh Manager container:

```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ sudo docker exec -it single-node-wazuh.manager-1 bash
```

Internal Rotation Configuration Monitoring:

The following command is used to check the internal options of the manager:

```
root@wazuh:/# cat /var/ossec/etc/internal_options.conf | grep -i "log\|rotate"
```

Output:

```
root@wazuh:/# cat /var/ossec/etc/internal_options.conf | grep -i "log\|rotate"
# Analysisd Enable the firewall log (at logs/firewall/firewall.log)
analysisd.log_fw=1
# Minimum output rotate interval. This limits rotation by time and size. [10..86400]
analysisd.min_rotate_interval=600
# Archives log queue size
# Statistical log queue size
# Alerts log queue size
# Firewall log queue size
# FTS log queue size
# Logcollector file loop timeout (check every 2 seconds for file changes)
logcollector.loop_timeout=2
# Logcollector number of attempts to open a log file [2..998] (0=infinite)
logcollector.open_attempts=0
# Logcollector - If it should accept remote commands from the manager
logcollector.remote_commands=0
# Logcollector - File checking interval (seconds) [0..1024]
logcollector.vcheck_files=64
# Logcollector - Maximum number of lines to read from the same file [100..1000000]
logcollector.max_lines=10000
# Logcollector - Maximum number of files to be monitored [1..100000]
logcollector.max_files=1000
logcollector.sock_fail_time=300
# Logcollector - Number of input threads for reading files
logcollector.input_threads=4
# Logcollector - Output queue size [128..220000]
logcollector.queue_size=1024
# Sample log length limit for errors about large message [1..4096]
logcollector.sample_log_length=64
# Maximum number of file descriptor that Logcollector can open [1024..1048576]
# This value must be higher than logcollector.max_files
logcollector.rlimit_nofile=1100
logcollector.force_reload=0
logcollector.reload_interval=64
```

```
logcollector.reload_delay=1000
logcollector.exclude_files_interval=86400
logcollector.state_interval=60
# Logbuilder IP update interval [0..3600]
logcollector.ip_update_interval=60
# Rotate plain and JSON logs daily. (0=no, 1=yes)
monitord.rotate_log=1
# Days to keep old ossec.log files [0..500]
monitord.keep_log_days=31
# Size of internal log files to rotate them (Megabytes) [0..4096]
monitord.size_rotate=512
# Maximum number of rotations per day for internal logs [1..256]
# Log collector (server, local or Unix agent)
logcollector.debug=0
```

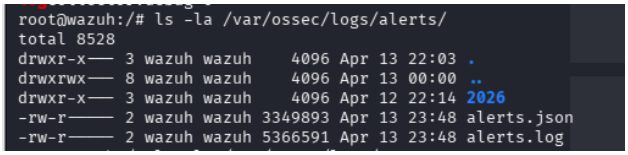
The output shows that *monitord.rotate_log=1* is enabled, which means that *Wazuh* rotates its internal log every day. Also, *monitord.keep_log_days=31* indicates that it keeps 31 days of logs.

Checking the alert directory (logs are sorted into folders by day/month)

Command:

ls -la /var/ossec/logs/alerts/

Output:

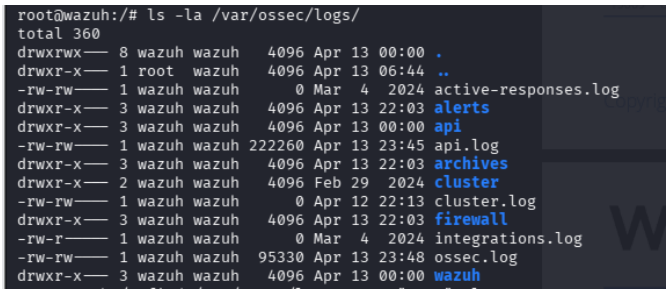


Here, I can see the *alerts.log* and *alerts.json* files which are the current live data. There is also a folder called 2026 where old logs are being stored.

Command:

ls -la /var/ossec/logs/

Output:

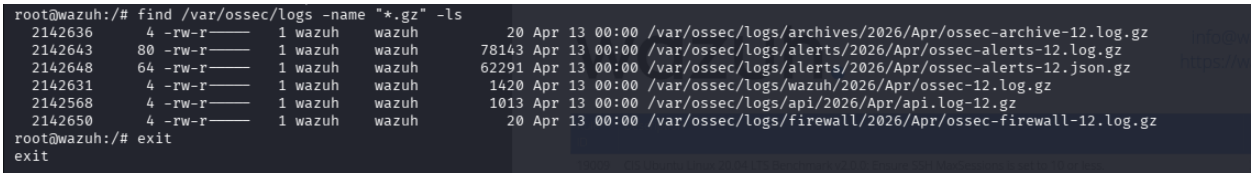


Compression (Archive) Verification:

To verify whether old logs are being zipped or compressed to save disk space:



Output:



I found that the old logs from *April* (*e.g. ossec-alerts-12.log.gz*) were successfully archived and compressed. This proves that the Wazoo system is capable of storage optimization. Then, *exit* the container by typing exit.

Creating a Custom Logrotate Config (Linux Agent)

A custom policy has been created to automatically manage the logs of a particular application at the endpoint level.

Creating a test environment:

First, a directory and test log files have been created:

```
osboxes@hasib:~$ sudo mkdir -p /var/log/wazuh-custom
```

Command:

echo "Wazuh test log entry $(date)" | sudo tee /var/log/wazuh-custom/test.log

Output:

```
osboxes@hasib:~$ echo "Wazuh test log entry $(date)" | sudo tee /var/log/wazuh-custom/test.log
Wazuh test log entry Tue Apr 14 02:50:17 AM EDT 2026
```

Custom rotation policy setup:

A new configuration file is created in the */etc/logrotate.d/* directory:

```
osboxes@hasib:~$ sudo nano /etc/logrotate.d/wazuh-custom
```

Configuration content:

```
  GNU nano 7.2                                          /etc/logrotate.d/wazuh-custom *
/var/log/wazuh-custom/*.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    sharedscripts
    postrotate
        /bin/kill -HUP `cat /var/run/syslogd.pid 2>/dev/null` 2>/dev/null || true
    endscript
}
```

Policy Analysis:

- **daily:** The log file will be rotated daily.
- **rotate 7:** The system will have a maximum of 7 old log files, if more than this, the old one will be deleted.
- **compress:** The old files will be compressed in *.gz* format to save storage.
- **postrotate:** After rotation, the system will reload the syslog service so that the log will start writing to the new file.

Log Rotation Testing and Verification (Practical Testing)

The main goal here is to ensure that the custom *wazuh-custom* configuration is rotating and compressing (archiving) the log files correctly.

Inputting data into the log (Log Population)

First, 50 dummy security events are added to the */var/log/wazuh-custom/test.log* file by running a loop.

Command:

```
osboxes@hasib:~$ for i in {1..50}; do
  echo "$(date) - Security Event $i - Test log entry" | sudo tee -a /var/log/wazuh-custom/test.log > /dev/null
done
```

The following command shows that the file size is *12 kilobytes.*

```
osboxes@hasib:~$ ls -lah /var/log/wazuh-custom/
total 12K
drwxr-xr-x  2 root root   4.0K Apr 14 02:50 .
drwxrwxr-x 19 root syslog 4.0K Apr 14 02:50 ..
-rw-r--r--  1 root root   3.5K Apr 14 03:28 test.log
```

Debugging and Dry-Run (Debug Mode)

Debug mode was run to see what log-rotate would actually change.

```
osboxes@hasib:~$ sudo logrotate -d /etc/logrotate.d/wazuh-custom
```

Output:

```
osboxes@hasib:~$ sudo logrotate -d /etc/logrotate.d/wazuh-custom
warning: logrotate in debug mode does nothing except printing debug messages!  Consider using verbose mode (-v) instead if this is not what you want.

reading config file /etc/logrotate.d/wazuh-custom
Reading state from file: /var/lib/logrotate/status
Allocating hash table for state file, size 64 entries
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
```

```
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state

Handling 1 logs

rotating pattern: /var/log/wazuh-custom/*.log  after 1 days (7 rotations)
empty log files are not rotated, old logs are removed
considering log /var/log/wazuh-custom/test.log
Creating new state
  Now: 2026-04-14 03:28
  Last rotated at 2026-04-14 03:00
  log does not need rotating (log has already been rotated)
not running postrotate script, since no logs were rotated
```

The output shows that the system has detected the test.log file and is ready for rotation.

Performing a manual rotation (Forced Rotation)

Since *Linux* usually rotates the log after a certain period of time, I used the *-f (Force*) flag to manually test it.

1st rotation:

```
osboxes@hasib:~$ sudo logrotate -f /etc/logrotate.d/wazuh-custom
```

The output shows that the *test.log* file has been renamed to *test.log.1*.

2nd rotation and compression:

```
osboxes@hasib:~$ echo "New entry after rotation" | sudo tee /var/log/wazuh-custom/test.log
sudo logrotate -f /etc/logrotate.d/wazuh-custom
ls -lah /var/log/wazuh-custom/
```

Output:

```
osboxes@hasib:~$ echo "New entry after rotation" | sudo tee /var/log/wazuh-custom/test.log
sudo logrotate -f /etc/logrotate.d/wazuh-custom
ls -lah /var/log/wazuh-custom/
New entry after rotation
total 16K
drwxr-xr-x  2 root root   4.0K Apr 14 03:29 .
drwxrwxr-x 19 root syslog 4.0K Apr 14 02:50 ..
-rw-r--r--  1 root root     25 Apr 14 03:29 test.log.1
-rw-r--r--  1 root root    247 Apr 14 03:28 test.log.2.gz
```

The output shows that the *test.log.1* file has been compressed into *test.log.2.gz* and a new *test.log.1* has been created.

Automation and Status Verification (System Integration)

The following files have been checked to ensure that the system is running it automatically on a daily basis.

1. Cron Job Verification:

Command:

```
osboxes@hasib:~$ cat /etc/cron.daily/logrotate
```

Output:

```
osboxes@hasib:~$ cat /etc/cron.daily/logrotate
#!/bin/sh

# skip in favour of systemd timer
if [ -d /run/systemd/system ]; then
    exit 0
fi

# this cronjob persists removals (but not purges)
if [ ! -x /usr/sbin/logrotate ]; then
    exit 0
fi

/usr/sbin/logrotate /etc/logrotate.conf
EXITVALUE=$?
if [ $EXITVALUE != 0 ]; then
    /usr/bin/logger -t logrotate "ALERT exited abnormally with [$EXITVALUE]"
fi
exit $EXITVALUE
```

This script ensures that the system automatically runs the *logrotate* command every day.

2. Rotation status check:

Command:

```
osboxes@hasib:~$ sudo cat /var/lib/logrotate/status | head -20
```

output:

```
osboxes@hasib:~$ sudo cat /var/lib/logrotate/status | head -20
logrotate state -- version 2
"/var/log/syslog" 2026-4-12-16:27:47
"/var/log/dpkg.log" 2026-4-12-16:27:47
"/var/log/speech-dispatcher/debug-flite" 2026-4-13-1:0:0
"/var/log/unattended-upgrades/unattended-upgrades.log" 2026-4-13-1:0:0
"/var/log/auth.log" 2026-4-12-16:27:47
"/var/log/wazuh-custom/test.log" 2026-4-14-3:29:30
"/var/log/apt/term.log" 2026-4-12-16:27:47
"/var/log/ppp-connect-errors" 2026-4-13-1:0:0
"/var/log/speech-dispatcher/speech-dispatcher-protocol.log" 2026-4-13-1:0:0
"/var/log/apt/history.log" 2026-4-12-16:27:47
"/var/log/boot.log" 2026-4-14-0:0:0
"/var/log/alternatives.log" 2026-4-12-16:27:47
"/var/log/sssd/*.log" 2026-4-13-1:0:0
"/var/log/mail.log" 2026-4-13-1:0:0
"/var/log/kern.log" 2026-4-12-16:27:47
"/var/log/cups/access_log" 2026-4-14-0:0:0
"/var/log/ufw.log" 2026-4-13-1:0:0
"/var/log/speech-dispatcher/speech-dispatcher.log" 2026-4-13-1:0:0
"/var/log/ubuntu-advantage.log" 2026-4-12-16:27:47
```

The output shows that "*/var/log/wazuh-custom/test.log*" *2026-4-14-3:29:30* This proves that the system has successfully recorded the rotation time of the custom log.

I have successfully implemented a custom log rotation policy which ensures the following:

- **Disk Efficiency:** Old logs are compressed (*.gz*) to save disk space.
- **Data Integrity:** Logs are stored serially *(1, 2, 3...)* without being lost.
- **Automation:** The system's *Cron Job* and status file ensure that it will work automatically every day.

Common Issues and Solutions

Issue: Container not starting / showing "Unhealthy" and Agent "Never Connected"

Troubleshooting and Connectivity Verification

The following steps were performed to ensure Docker container performance and agent-manager connectivity during the lab:

Step 1: Wazuh Indexer Memory Fix (Wazuh Server)

In order to run *Wazuh* Indexer (based on *Elasticsearch*), it was necessary to increase the virtual memory limit of the *Linux kernel*. Failure to do this may cause the indexer container to show "*Unhealthy*".

Increasing the Memory Limit

Command:

echo "vm.max_map_count=262144" | sudo tee -a /etc/sysctl.conf

sudo sysctl -p

Output:

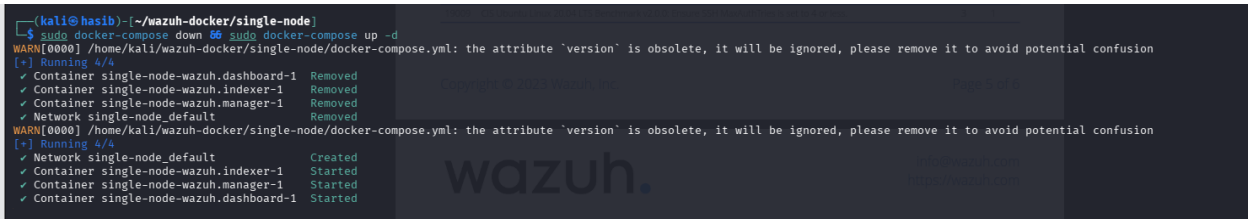
```
┌──(kali㉿hasib)-[~/wazuh-docker/single-node]
└─$ echo "vm.max_map_count=262144" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
[sudo] password for kali:
vm.max_map_count=262144
vm.max_map_count = 262144
```

Restarting the container

Command:

sudo docker-compose down && sudo docker-compose up -d

Output:



The Docker containers restarted successfully after updating the memory configuration.

Step 2: Monitoring Agent Status and Logs (Linux Agent)

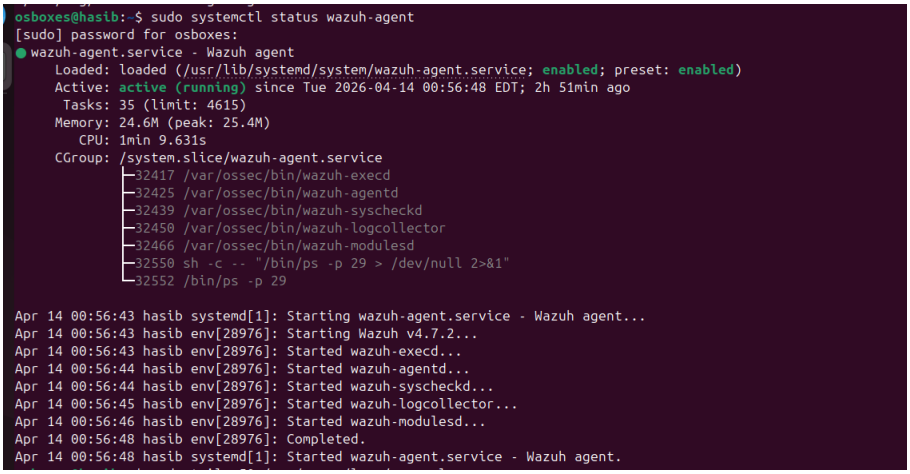
The following commands were run on the *Ubuntu VM* to ensure that the agent is working properly:

Viewing the current status of the agent

Command:

sudo systemctl status wazuh-agent

Output:

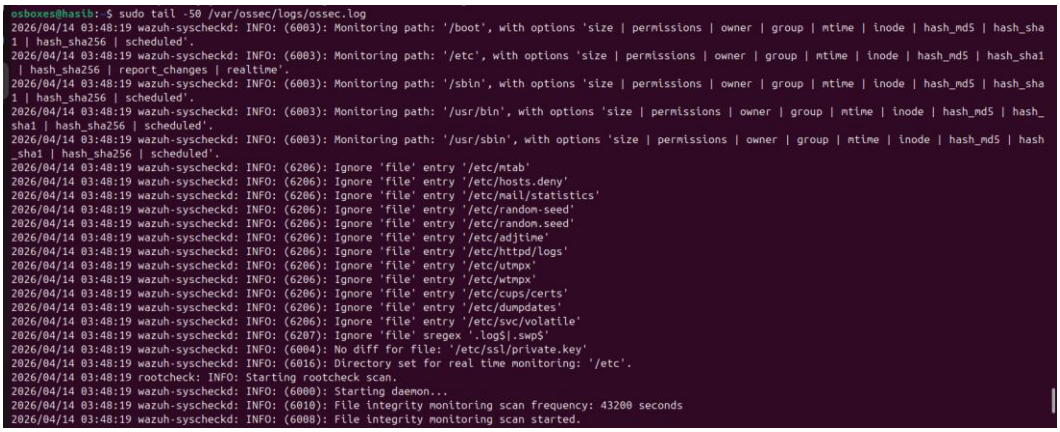


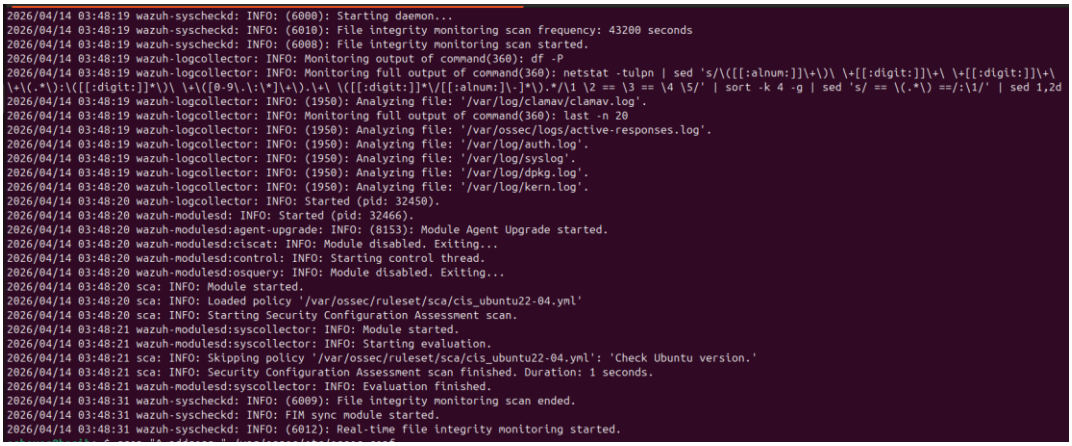
View live log files

Command:

sudo tail -50 /var/ossec/logs/ossec.log

Output:





The output shows that the agent is active (*running*) and the *Syscheck* and *Log-collector* modules have been successfully started.

Step 3: Verifying IP Address and Configuration (Connectivity Discovery)

The agent is trying to connect to which manager and the *Ubuntu VM's* own *IP* has been verified.

Net-tools Installation:

The *ifconfig* command was required to view the IP.

Command:

sudo apt install net-tools

Output:

```
osboxes@hasib:~$ sudo apt install net-tools
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  net-tools
0 upgraded, 1 newly installed, 0 to remove and 619 not upgraded.
Need to get 204 kB of archives.
After this operation, 811 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 net-tools amd64 2.10-0.1ubuntu4.4 [204 kB]
Fetched 204 kB in 1s (180 kB/s)
Selecting previously unselected package net-tools.
(Reading database ... 154063 files and directories currently installed.)
Preparing to unpack .../net-tools_2.10-0.1ubuntu4.4_amd64.deb ...
Unpacking net-tools (2.10-0.1ubuntu4.4) ...
Setting up net-tools (2.10-0.1ubuntu4.4) ...
Processing triggers for man-db (2.12.0-4build2) ...
```

Ifconfig

```
osboxes@hasib:~$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether ea:e3:df:64:4c:ec  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.91.134  netmask 255.255.255.0  broadcast 192.168.91.255
        inet6 fe80::f6a9:64ea:168b:40bb  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:11:4a:da  txqueuelen 1000  (Ethernet)
        RX packets 1569131  bytes 2281902182 (2.2 GB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 96642  bytes 13237461 (13.2 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 404  bytes 50961 (50.9 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 404  bytes 50961 (50.9 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Ubuntu VM (Linux Agent) IP: 192.168.91.134

Manager IP Verification:

Command:

sudo grep -i "<address>" /var/ossec/etc/ossec.conf

Output:

```
osboxes@hasib:~$ sudo grep -i "<address>" /var/ossec/etc/ossec.conf
      <address>192.168.91.132</address>
```

The output shows that the *IP* of the manager is given in the configuration file as *192.168.91.132*.

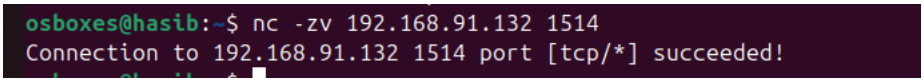
Step 4: Network Port Test

It has been checked whether the specified port *(1514)* of the manager *(Kali VM)* can be contacted from the agent *(Ubuntu)*.

Command:

nc -zv 192.168.91.132 1514

Output:



The output shows that *Connection to 192.168.91.132 1514 port [tcp/*] succeeded!*

This proves that there is no firewall issue at the network level and the agent is able to successfully communicate with the manager.

References

[1] Wazuh, "Wazuh Documentation: Installation guide," *Wazuh.com*, 2024. Available: https://documentation.wazuh.com/current/installation-guide/index.html

[2] Docker, "Docker Desktop and Engine documentation," *Docker Docs*, 2024. Available: https://docs.docker.com/get-started/

[3] MITRE ATT&CK, "Brute Force, Technique T1110," *MITRE ATT&CK Framework*, 2024. Available: https://attack.mitre.org/techniques/T1110/

[4] Canonical, "Ubuntu Server Guide: Security - OpenSSH," *Ubuntu.com*, 2024. Available: https://ubuntu.com/server/docs/service-openssh

[5] Wazuh, "Wazuh Documentation: File Integrity Monitoring (FIM)," *Wazuh.com*, 2024. Available: https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity-monitoring/index.html

[6] THC Hydra, "Hydra: A very fast network logon cracker," *GitHub*, 2023. Available: https://github.com/vanhauser-thc/thc-hydra

[7] Oracle, "Oracle VM VirtualBox User Manual," *VirtualBox.org*, 2024. Available: https://www.virtualbox.org/manual/ch01.html

[8] Wazuh, “Wazuh Documentation: Active Response,” *Wazuh.com*, 2024. Available: https://documentation.wazuh.com/current/user-manual/capabilities/active-response/index.html

[9] ClamAV, “Clam Antivirus Documentation,” *ClamAV.net*, 2024. Available: https://docs.clamav.net/

[10] Elastic, “Elasticsearch, Logstash, and Kibana (ELK) Stack Overview,” *Elastic.co*, 2024. Available: https://www.elastic.co/what-is/elk-stack

[11] Wazuh, “Wazuh Docker Deployment - GitHub Repository,” *GitHub*, 2024. Available: https://github.com/wazuh/wazuh-docker

[12] Microsoft, “MsiExec.exe Command-Line Options,” *Microsoft Learn*, 2023. Available: https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/msiexec

[13] EICAR, “The Antivirus Test File,” *EICAR.org*, 2024. Available: https://www.eicar.org/download-anti-malware-testfile/